ulm.lam: Write the Formula, Get the Expert's BLAS and LAPACK Code

Ulm Linear Algebra Mapping with MLIR and ulmBLAS

Design draft

November 12, 2026

Abstract

You write linear algebra as on paper, in Python or Julia, with inverses, transposes and matrix properties; ulm.lam compiles it into the code an expert in numerical linear algebra would write on top of BLAS (Basic Linear Algebra Subprograms) and LAPACK (Linear Algebra PACKage): factorisations instead of inverses, computed once and pulled out of loops, products parenthesised by cost, symmetric results computed as one triangle, no memory allocated inside nested calls, and errors reported as LAPACK's `info`. The prediction of a Gaussian process, five lines of Python, runs 1.4× faster than the same steps written by hand with SciPy and 8× faster than the formula written in NumPy, and scales to $n = 100\,000$ on a cluster node, where it is as fast as the same steps written by hand with Intel's MKL (Math Kernel Library). A Kalman filter loop, written as the formula, is 3× faster than NumPy code optimised by hand. Behind it is a solution of the *Linear Algebra Mapping Problem* (LAMP) on MLIR (Multi-Level Intermediate Representation) [8]: two dialects (the algebra with matrix properties, and the kernels in destination-passing style), kernel selection by a generalised matrix chain algorithm in the spirit of Linnea [2, 3], MLIR's bufferization for the memory, and a lowering to C++ calling ulmBLAS [16], or any CBLAS and LAPACK.

Contents

1 ulm.lam at a glance **2**

2 The linear algebra mapping problem **5**

3 Related work **6**

4 Architecture **8**

5 The `ulm_lam` dialect **9**

6 The `ulm_lamk` dialect **10**

7 Kernel selection **11**

8 Bufferization and lowering **14**

9 Worked example: generalised least squares **20**

10 Formula (1) in a loop: a Kalman measurement update **25**

11 Python frontend **27**

12 Julia frontend **29**

13 Showcase: Gaussian process regression **30**

14 Testing strategy **32**

15 Status and plan **32**

A Generated code **34**

1 ulm.lam at a glance

What you write. The prediction of a Gaussian process (GP), the textbook formula

$$\mu = K_*^{\mathsf{T}}(K + \sigma^2 I)^{-1} y, \qquad \Sigma = K_{**} - K_*^{\mathsf{T}}(K + \sigma^2 I)^{-1} K_*,$$

written in Python:

```
from ulm import lam
from ulm.lam import Float, Matrix, Vector

@lam.function
def gp(K: Matrix("n", "n", "symmetric"), s2: Float, Ks: Matrix("n", "m"),
       Kss: Matrix("m", "m", "symmetric"), y: Vector("n")):
    Kn = lam.assume(K + s2 * lam.eye("n"), "spd")
    return Ks.T @ Kn.inv @ y, Kss - Ks.T @ Kn.inv @ Ks

lib = lam.compile(gp)                  # C++ calling ulmBLAS, compiled once
mean, cov = lib.gp(K, 0.01, Ks, Kss, y)  # NumPy arrays in and out
```

The sizes are names: one compiled library serves all n and m. `lam.assume` states what the compiler cannot know ($K + \sigma^2 I$ is positive definite for $\sigma^2 > 0$); the Cholesky factorisation checks it at run time.

How fast it is. Bike-sharing data (UCI, University of California, Irvine), $m = 1000$ test points, Apple M5 (seconds, without computing the kernel matrices):

n	ulm.lam/ulmBLAS	ulm.lam/OpenBLAS	SciPy by hand	scikit-learn	NumPy literal
2000	0.036	0.046	0.053	0.119	0.194
4000	0.133	0.189	0.242	0.547	1.109
8000	0.886	1.105	1.209	2.970	6.983

"SciPy by hand" is what scikit-learn does inside, written directly with SciPy; "NumPy literal" is the formula with `np.linalg.inv`; scikit-learn's time includes the kernel matrices, which it computes inside (0.78 s of its 2.97 s at $n = 8000$). On the same BLAS (OpenBLAS, the one NumPy uses here) the generated code is faster than the hand-written version; on ulmBLAS it is faster still. On a cluster node (2× AMD EPYC 9454, 96 cores) the same function predicts terrain altitudes from $n = 100\,000$ road points (Section 13.1): the factorisation of an 80 GB matrix, in 154 s on MKL, as fast as the same steps written by hand with MKL. And in a loop, the Kalman measurement update $b_i = (H^{\mathsf{T}} R^{-1} H)^{-1} H^{\mathsf{T}} R^{-1} y_i$ for 200 measurements ($n = 4000$) takes 122 ms written as the formula, against 368 ms for NumPy code in which the operator was formed by hand before the loop, and 270 s for the formula written in NumPy (Section 11).

The same in Julia. The Julia package (Section 12) uses Julia's notation; Julia's column-major arrays are passed without copies:

```
using UlmLam, LinearAlgebra

@lam function gp(K::Mat(:n, :n, :symmetric), s2::Float, Ks::Mat(:n, :m),
```

```
                Kss::Mat(:m, :m, :symmetric), y::Vec(:n))
    Kn = assume(K + s2 * I, :spd)
    (Ks' * inv(Kn) * y, Kss - Ks' * inv(Kn) * Ks)
end

lib = UlmLam.compile(gp)
mean, cov = lib.gp(K, 0.01, Ks, Kss, y)
```

Compared with what a Julia programmer writes by hand (`C = cholesky(Symmetric(K + s2*I))`, `C \textbackslash {} y`, `V = C.L \textbackslash {} Ks`, `Kss - V'V`) and with the formula written literally (`inv`), RBF (radial basis function) kernel matrices of random points, $m = 1000$, Apple M5, Julia 1.13 with its OpenBLAS on 4 threads (its default here; seconds):

n	ulm.lam/ulmBLAS	ulm.lam/OpenBLAS	Julia by hand	Julia literal
2000	0.037	0.045	0.071	0.212
4000	0.137	0.216	0.332	1.359
8000	0.907	1.462	1.922	8.828

In the Kalman loop (200 steps, $n = 4000$), ulm.lam takes about 140 ms, Julia with the operator formed by hand before the loop 390 to 550 ms (it varies between runs), and the formula written in Julia 400 to 520 s (`julia/examples/kalman.jl`).

Why it is fast. ulm.lam is a compiler: the formula is traced into an intermediate representation, mapped onto BLAS and LAPACK calls, and compiled to C++. The mapping makes the decisions an expert makes when writing such code by hand:

- inverses become factorisations and triangular solves; a matrix is factorised once, however often its inverse appears;
- properties are exploited: Cholesky instead of LU for positive definite matrices, `syrk` for symmetric products (half the flops of `gemm`), one triangle of symmetric matrices;
- chains of products are parenthesised to minimise the cost, at run time if the best order depends on the sizes;
- factorisations and other loop-invariant work are pulled out of loops, and the whole operator is formed before a loop if the number of steps makes it pay off;
- no memory is allocated dynamically, not even inside nested calls: all temporaries live in one workspace with a LAPACK-style size query, kernels work in place where an input is no longer needed, and errors are returned as LAPACK's `info`.

In short: it generates the code a programmer experienced in numerical linear algebra would write on top of BLAS and LAPACK.

What the mapping does. With $K + \sigma^2 I = LL^{\mathsf{T}}$ (Cholesky, L lower triangular), $(K + \sigma^2 I)^{-1} = L^{-\mathsf{T}} L^{-1}$, so

$$\mu = K_*^{\mathsf{T}} \alpha \quad \text{with} \quad \alpha = L^{-\mathsf{T}}(L^{-1} y), \qquad \Sigma = K_{**} - V^{\mathsf{T}} V \quad \text{with} \quad V = L^{-1} K_*.$$

One factorisation serves both results, no inverse is formed, and $V^{\mathsf{T}} V$ is symmetric, so only one triangle is computed. As calls (all temporaries in the workspace w):

	operation	kernel
1	$W \leftarrow K,\ W \leftarrow W + \sigma^2 I$	copy, `adddiag`
2	$W = LL^{\mathsf{T}}$ (L overwrites W); if it fails: return `info` 1	`potrf`
3	$a \leftarrow y,\ a \leftarrow L^{-1}a,\ a \leftarrow L^{-\mathsf{T}}a$	copy, `trsv`, `trsv`
4	$\mu \leftarrow K_*^{\mathsf{T}} a$	`gemv`
5	$V \leftarrow K_*,\ V \leftarrow L^{-1}V$	copy, `trsm`
6	$\Sigma \leftarrow K_{**},\ \Sigma \leftarrow \Sigma - V^{\mathsf{T}}V$ (lower triangle), mirror it	copy, `syrk`, complete

The copies exist because K, K_* and K_{**} are `in` parameters, which the function must not overwrite; declared `InOut`, K is factorised in place (Section 13.1). Figure 1 is the C++ that ulm.lam generates from the five lines of Python.

```
/* gp: matrices by reference (row-major), owned by the caller
 *   v1   size_t          in
 *   v2   size_t          in
 *   v3   double[v1][v1]  in    read only
 *   v4   double          in
 *   v5   double[v1][v2]  in    read only
 *   v6   double[v2][v2]  in    read only
 *   v7   double[v1][1]   in    read only
 *   v8   double[v2][1]   out   written
 *   v9   double[v2][v2]  out   written
 *   v10  int8_t *        work  workspace (bytes)
 *   v11  int64_t *       info  -1: query, 0: allocate, >0: size of work
 * returns info:
 *      0  success
 *      1  Cholesky factorisation of a matrix computed from v3: not positive
 *         definite
 *    -11  v11: workspace too small
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
gp(size_t v1, size_t v2, double *v3, double v4, double *v5, double *v6,
   double *v7, double *v8, double *v9, int8_t *v10, int64_t *v11)
{
    double v12 = -1.0000000000000000e+00;
    double v13 = 1.0000000000000000e+00;
    double v14 = 0.0e+00;
    int64_t v15 = 0;
    size_t v16 = -64;
    size_t v17 = 63;
    size_t v18 = 0;
    int64_t v19 = v11[v18];
    size_t v20 = v1 * 8;
    size_t v21 = v20 * v1 + v17 & v16;
    size_t v22 = v21 + (v20 + v17 & v16);
    size_t v23 = v22 + (v20 * v2 + v17 & v16);
    ptrdiff_t v24 = (ptrdiff_t)v23;
    int64_t v25 = (int64_t)v24;
    int64_t v26;
    if (v19 == -1) {
        v11[v18] = v25;
        v26 = v15;
    } else {
        int64_t v27;
        if (v19 > v15 & v19 < v25) {
            v27 = -11;
        } else {
            bool v28 = v19 == v15;
            int8_t *v29;
            if (v28) {
```

```
                void *v30 = aligned_alloc(64, v23);
                v29 = (int8_t *)v30;
            } else {
                v29 = v10;
            }
            int8_t *v31 = v29;
            int64_t v32 = (int64_t)v18;
            double *v33 = (double *)(v31 + v32);
            size_t v34 = (8 * v1) * v1;
            ulm_lam_copy(v33, v3, v34);
            ulm_lam_adddiag(v1, v4, v33, v1, 1);
            int64_t v35 = ulmBLAS::potrf<std::ptrdiff_t>(true, v1, v33, v1, 1);
            bool v36 = (bool)(v35 != v15);
            uint64_t v37 = (uint64_t)v36;
            int64_t v38 = (int64_t)v37;
            if (v38 == v15) {
                int64_t v39 = (int64_t)v21;
                double *v40 = (double *)(v31 + v39);
                size_t v41 = 8 * v1;
                ulm_lam_copy(v40, v7, v41);
                ulmBLAS::trlsv<std::ptrdiff_t>(v1, false, v33, v1, 1, v40, 1);
                ulmBLAS::trusv<std::ptrdiff_t>(v1, false, v33, 1, v1, v40, 1);
                ulmBLAS::gemv<std::ptrdiff_t>(v2, v1, v13, v5, 1, v2, v40, 1,
                                              v14, v8, 1);
                int64_t v42 = (int64_t)v22;
                double *v43 = (double *)(v31 + v42);
                size_t v44 = (8 * v1) * v2;
                ulm_lam_copy(v43, v5, v44);
                (void)ulmBLAS::trtrs<std::ptrdiff_t>(
                    true, false, false, false, v1, v2, v33, v1, 1, v43, v2, 1);
                size_t v45 = (8 * v2) * v2;
                ulm_lam_copy(v9, v6, v45);
                ulmBLAS::sylrk<std::ptrdiff_t>(v2, v1, v12, v43, 1, v2, v13, v9,
                                               v2, 1);
                ulm_lam_sycomplete(true, v2, v9, v2, 1);
            }
            if (v28) {
                free(v31);
            }
            v27 = v38;
        }
        v26 = v27;
    }
    return v26;
}
```

Figure 1: The function `gp` generated from the Python above (`python/examples/generated/gp.cpp`, without the `#include` lines and the helpers `ulm_lam_copy`, `ulm_lam_adddiag` and `ulm_lam_sycomplete` in front of it). The interface comment says what the caller passes and what `info` means; the outer `if`s implement the workspace query, the inner ones `info`; then come the steps of the table above.

2 The linear algebra mapping problem

A linear algebra problem is written as mathematics:

$$b \;=\; (X^{\mathsf{T}} M^{-1} X)^{-1} X^{\mathsf{T}} M^{-1} y, \qquad M \text{ symmetric positive definite (SPD)},\; X \text{ of full rank}, \tag{1}$$

the generalised least-squares estimator. It is executed as a sequence of calls to tuned kernels: `potrf`, `trsm`, `syrk`, `gemv`, `trsv`. Choosing that sequence is the LAMP [1]. The choice includes parenthesisations, which inverses become solves, which factorisations to compute, which properties to exploit, and which

subexpressions to share. The problem is NP-complete (NP: nondeterministic polynomial time) in general [1]. Mainstream languages and libraries (Matlab, Julia, Eigen, Armadillo, NumPy, TensorFlow, PyTorch) solve it only partially [1, 6].

Linnea [2, 3] is the reference solution. It normalises the input by term rewriting with MatchPy [5], infers matrix properties, and runs a best-first search over a graph of derivations. It emits Julia code that calls BLAS/LAPACK and reaches expert-level performance. It is also a Python research prototype, last changed in 2021.

Why MLIR. MLIR provides what Linnea had to build by hand, and much that it lacks:

- a typed, verified intermediate representation (IR) and a textual format that are stable;
- rewrite infrastructure, common subexpression elimination (CSE), and `lit`/`FileCheck` testing;
- One-Shot Bufferize, which decides in-place updates and copies. This replaces Linnea's hand-written "overwriting" logic;
- lowering to LLVM (originally "Low Level Virtual Machine", today just the project's name), and a code-generation fallback (`linalg`) when no library kernel fits;
- frontends, Python bindings and an ecosystem that is maintained.

What MLIR does not provide. The core of the LAMP is not covered by any upstream component:

- **search instead of greedy rewriting.** MLIR's pattern driver applies rewrites greedily. Kernel selection must compare alternatives under a cost model;
- **associative-commutative matching.** MLIR patterns match DAGs (directed acyclic graphs) structurally, while kernels have to be found inside products and sums of any length;
- **a dialect at the right level.** `linalg` has no inverse, no notion of SPD, and no transpose as an algebraic operator.

The design below keeps these parts explicit and small, and uses upstream MLIR for everything else.

What this repository contains. Table 1 lists the files. The ODS files are *checked*: `scripts/check-ods.sh` runs every `mlir-tblgen` backend the dialect needs, including validation of the custom assembly formats. The dialects are *implemented*: `lib/` contains both dialects, generated from the ODS, and a first set of passes, built as a plugin for `mlir-opt` (Section 7.4). The worked example is partly automatic. Its stages 1–4 are written by hand as a design: they show Linnea's derivation. They parse and verify with the implemented dialects, and stage 3 bufferizes into exactly the copies that stage 4 shows. From stage 2 on, the passes produce C++ calling ulmBLAS by themselves; it is compiled, run and verified numerically (Section 9). The loop example (Section 8.1) and eight kernel selection cases with inverses, solves and sums are compiled the same way and checked against naive evaluations.

3 Related work

Linnea. Linnea [2, 3] works in four steps:

1. it rewrites the input into a normal form, with associative operators flattened, transposes and inverses pushed to the leaves, and properties propagated;
2. it represents the program as a graph of derivations, where each edge applies a kernel or a factorisation;

3. it explores that graph best-first, with FLOPs (floating-point operations) as cost. The search is *anytime*: a first solution typically comes in under a second, and better ones follow if it runs longer;
4. it finds matrix products with the generalised matrix chain (GMC) algorithm [4].

GMC is a dynamic programme over parenthesisations. It chooses a kernel for every binary product and takes transposes, inverses and properties into account. Matching uses MatchPy's associative-commutative matching with sequence variables [5]. The implementation is Python. Its kernel definitions exist for Julia and C, and it generates Julia (see `https://github.com/HPAC/linnea`).

MOM / MLIR-Linnea. Chelini et al. [7] proposed a matrix dialect for MLIR that carries properties in the types, has n-ary multiplication, and contains a generalised matrix-chain pass. It lowers to `linalg`. The prototype[1] has property propagation and matrix-chain reordering. Kernel detection is stubbed: `isTRSM()` returns `false`. It has no search, and it is pinned to a 2022 LLVM. This sketch follows MOM in putting properties into types. It differs in three ways:

- it uses the builtin tensor type with an encoding, as `sparse_tensor` does, instead of a new type;
- it adds a kernel dialect in destination-passing style, so that upstream bufferization can do the memory planning;
- it makes the search a separate, explicit component.

Equality saturation. Equality saturation (egg [9], egglog [10]) explores all rewrites at once and extracts a cheapest term. SPORES [11] applies it to linear algebra in a relational setting. Two MLIR integrations exist:

- the `eqsat` dialect [12], which represents an e-graph inside MLIR and is driven by PDL (Pattern Description Language);
- DialEgg [13], which translates any MLIR dialect to egglog and back.

Neither has been applied to the LAMP. They are the natural alternative to a hand-written search (Section 7.6).

Library calls from MLIR. LAPIS [17] lowers MLIR linear algebra to Kokkos Kernels calls. That is the "op → library call" step, but it does not use matrix properties.

FLENS. FLENS [14, 15] pursued the same goal, a convenient notation with an efficient evaluation, as a header-only C++ library. Overloaded operators build closures that are evaluated by BLAS calls: an assignment such as `y = beta*y + alpha*transpose(A)*x + z` becomes one `mv` (`gemv`) and one `axpy`, which inlining turns into direct calls of the BLAS. Matrix properties are part of the types (`GeMatrix`, `SyMatrix`, `TrMatrix`), and they select the kernel: `mv` is `gemv`, `symv` or `trmv`, `mm` is `gemm`, `symm` or `trmm`. By default FLENS never creates a temporary: an expression that BLAS cannot evaluate without one is a compile-time error, or a run-time assertion where the compiler cannot decide it. LAPACK functionality is available through FLENS-LAPACK, called explicitly (e.g. `lapack::sv` for a linear system).

FLENS also shows where a library has to stop. Expression templates see one statement at a time, at the compile time of C++, and matrix sizes only at run time. They can map a fixed expression structure onto a fixed call, but they cannot choose a parenthesisation by cost, replace an inverse by a factorisation and share it, infer the properties of intermediate results, find common subexpressions across statements, move invariant work out of loops, or plan the memory of a whole function. Those decisions need the whole program as data, the sizes and a cost model: a compiler. ulm.lam keeps the principles of FLENS

[1] `https://github.com/Algebraic-Programming/MLIR-Linnea`, last commit June 2022.

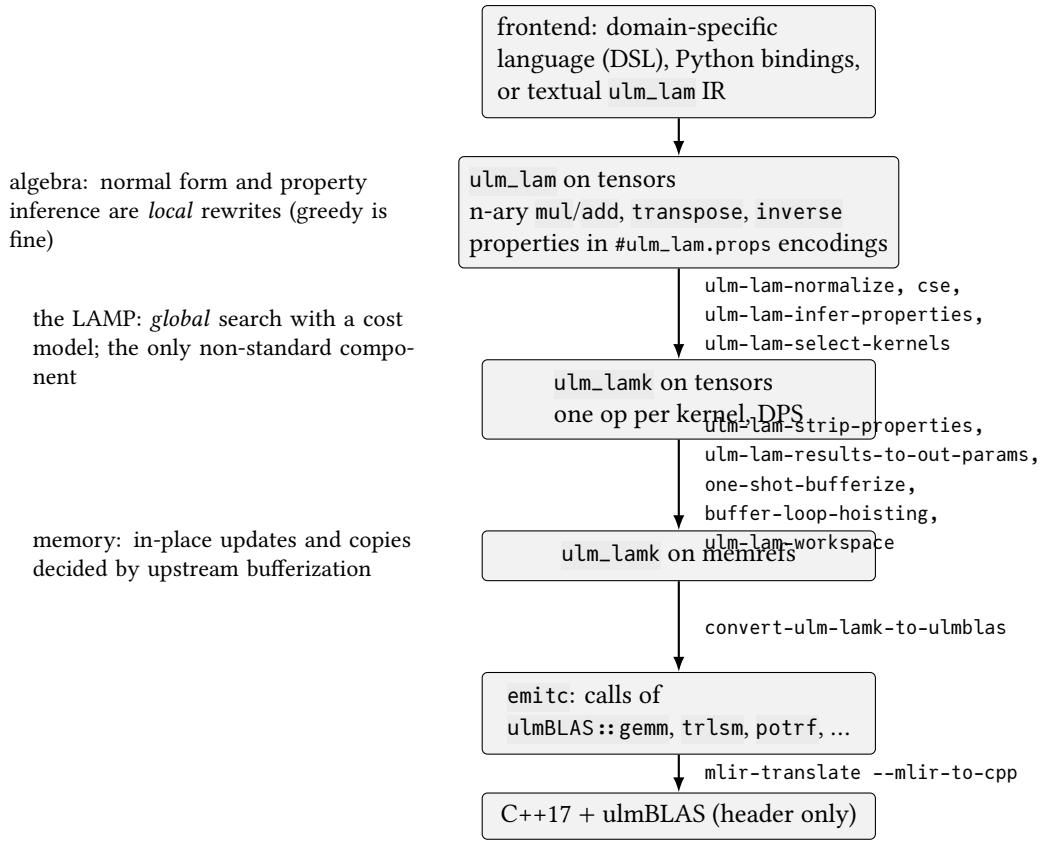


Figure 2: The pipeline. Everything below `ulm_lamk` on tensors is upstream MLIR plus two small passes.

and moves them there: properties in types become `#ulm_lam.props`; "no hidden temporaries" becomes the caller-owned workspace with its query (Section 8.1) and the parameter intents that say what a function overwrites; the notation becomes Python, traced at run time (Section 11); and the efficient evaluation is done by kernel selection and ulmBLAS.

FLENS-LAPACK: the work of a compiler, by hand. FLENS-LAPACK re-implemented a significant part of LAPACK on top of FLENS, bit for bit: compiled with `CHECK_CXXLAPACK`, every FLENS-LAPACK function also calls the reference LAPACK on copies of its arguments and requires identical results, compared by exact equality (`flens/lapack/debug/isidentical.tcc`); in addition it is checked with the Netlib LAPACK test suite (`flens/lapack/check-results.sh`, known deviations listed in `check-expected.txt`). The work was so mechanical that it felt like doing the job of a compiler: factorisations hoisted out of loops by hand, views of matrices mapped onto workspace arrays by hand, and the size of every workspace derived by hand through nested calls. It also showed what LAPACK requires to avoid temporaries deep inside nested calls: a workspace provided by the caller, a query that returns its size (accumulated over the callees), in-place kernels on strided views, and an `info` instead of an abort. These requirements are the conventions of the code ulm.lam generates (Section 8.1), and the mechanical work is what it automates: loop-invariant code motion and the operator chosen by the trip count (Section 10), one factorisation per matrix, and the workspace layout computed from the bufferized program. So ulm.lam is the compiler that FLENS-LAPACK was written without.

4 Architecture

Figure 2 shows the pipeline. It splits into three concerns, and each uses the MLIR mechanism that fits it:

Algebra (`ulm_lam`). Normal form and property inference are local and confluent. Canonicalisation

patterns and `InferTypeOpInterface` are the right tools.

Selection (`ulm_lam` → `ulm_lamk`). A global optimisation problem. One pass owns it and keeps its own search data structure (Section 7).

Memory and calls (`ulm_lamk` → C++). Destination-passing style lets One-Shot Bufferize plan buffers. Each kernel becomes one call of the ulmBLAS C++ interface.

Two dialects, rather than one, make the boundary checkable. After `ulm-lam-select-kernels`, a conversion target that marks `ulm_lam` illegal proves that every operation was mapped. Lowering passes never need to understand algebra.

5 The `ulm_lam` dialect

5.1 Values and properties

Every operand is a rank-2 builtin tensor:

- a column vector is $n \times 1$ and a row vector is $1 \times n$;
- scalars are plain floats;
- properties are a tensor *encoding*:

```
tensor<2000x2000xf64, #ulm_lam.props<spd>>
tensor<2000x200xf64,  #ulm_lam.props<full_rank>>
```

`sparse_tensor` attaches its storage format the same way. The benefits:

- standard tensor infrastructure (CSE, `tensor.empty`, function signatures, the parser and printer) works unchanged;
- properties show up in the textual IR, so tests can `FileCheck` them.

The encoding is a bit set (Fig. 3). Its meaning follows three rules:

- the set is an *assertion*, and a missing bit means "unknown", never "false". Dropping bits is therefore always sound;
- sets are read modulo an implication closure (`ulm_lam::closure`), e.g. spd ⇒ symmetric, non-singular;
- facts implied by a static shape (`square` on a 200×200 tensor) are not stored.

`full_rank` means rank $= \min(\text{rows}, \text{cols})$. For square matrices that is `non_singular`. The attribute implements `VerifiableTensorEncoding`, which rejects contradictions such as `zero` together with `non_singular`.

5.2 Property inference

Every op implements `InferTypeOpInterface`. Builders therefore compute result properties automatically, and every newly created value is typed as strongly as local reasoning allows. Table 2 shows the rules. They are local, but they look at operand *identity*, not only at operand properties. For example, $A^{\mathsf{T}}A$ is symmetric positive semi-definite (SPSD) only because both factors are the same SSA (static single assignment) value.

Relaxed type compatibility. By default, `InferTypeOpInterface` requires the result type to *equal* the inferred type. That rule would make rewriting fragile. Replacing an operand by a value with stronger properties would invalidate every user until its type was recomputed. The `ulm_lam` ops therefore override `isCompatibleReturnTypes`. A result type is accepted if its shape matches and its properties are *implied by* the inferred ones (Fig. 4). Weaker types are always valid. `ulm-lam-infer-properties` strengthens all types in one topological sweep. Facts that inference cannot derive enter only through function signatures or `ulm_lam.assume`. An assumption is also tested at run time where it matters: a Cholesky factorisation of a matrix that is assumed spd reports through `info` when the matrix is not positive definite numerically (Section 8.1).

5.3 Operations

Table 3 lists the operations. `ulm_lam.mul` is n-ary on purpose. A product denotes the mathematical object $A_1 \cdots A_k$. Choosing a parenthesisation is a decision of kernel selection, not of the frontend.

5.4 Normal form

`ulm-lam-normalize` applies canonicalisation patterns greedily and then runs CSE. The patterns are:

1. flatten nested `mul` and `add`;
2. push transposes to the leaves: $(AB)^\mathsf{T} \to B^\mathsf{T}A^\mathsf{T}$, $(A+B)^\mathsf{T} \to A^\mathsf{T} + B^\mathsf{T}$, $(A^\mathsf{T})^\mathsf{T} \to A$, and $A^\mathsf{T} \to A$ for symmetric A;
3. push inverses to the leaves for square non-singular factors: $(AB)^{-1} \to B^{-1}A^{-1}$ and $(A^\mathsf{T})^{-1} \to (A^{-1})^\mathsf{T}$. A leaf is then A, A^T, A^{-1} or $A^{-\mathsf{T}}$;
4. hoist scales out of products and merge them;
5. remove identities from products and zeros from sums; a product with a zero factor is zero.

Each pattern strictly decreases a well-founded measure: the depth of transpose and inverse nodes, and the nesting of `mul` and `add`. The driver therefore terminates. Normalising is a precondition for inference. In Section 9, the frontend's binary product $(X^\mathsf{T}M^{-1})X$ is not recognised as spd. The flattened product $X^\mathsf{T}M^{-1}X$ is.

The normal form is not unique for the purposes of the LAMP. Distributing a product over a sum can help or hurt, and so can factoring one out. Such rewrites are therefore *not* canonicalisations. They are moves of the search.

6 The `ulm_lamk` dialect

`ulm_lamk` has one operation per kernel (Table 4), with BLAS flags as enum attributes. The design follows four rules.

Destination-passing style. The operand that BLAS overwrites is the `outs` operand: C for `gemm`, B for `trsm`, A for `potrf`. On tensors, the op returns the updated value. On memrefs, it returns nothing. This is the contract of `DestinationStyleOpInterface`. It is exactly what One-Shot Bufferize needs to update `outs` buffers in place and to copy only when an overwritten value is still live (Section 8).

Precise partial results. `potrf` and `syrk` define only one triangle. On tensors, their semantics are those of BLAS: the `uplo` triangle receives the result, and the strict other triangle keeps the init's values. With a `tensor.empty` init those values are undefined. Kernel selection guarantees that every consumer reads only the defined triangle: the factor of `potrf` is only used by triangular solves with the same `uplo`, and the result of `syrk` is passed on as a matrix stored in its lower triangle (next paragraph).

Symmetric matrices stored as one triangle. By default a matrix is stored completely, also a symmetric one, and the generated code may read every element (e.g. with `gemm`). As with the `uplo` argument of LAPACK, a parameter can instead be declared to be stored as one triangle: `Matrix(n, n, "spd", stored="lower")`, in the encoding `#ulm_lam.props<spd, lower_stored>`. The other triangle may then hold anything. Unlike the other bits of the encoding, these two are not assertions about the value but about its memory, so dropping them is *not* sound. They are never inferred and never propagated to the result of an op; kernel selection resolves every use:

- a product with such a factor uses `symm` (ulmBLAS `sylmm`/`syumm`; the right side BA is the left one on transposed views, $C^\mathsf{T} = AB^\mathsf{T}$) or `symv` (`sylmv`; for the upper triangle on the transposed view, because $A^\mathsf{T} = A$), never `gemm`/`gemv`;
- a Cholesky factorisation uses the stored triangle: $A = LL^\mathsf{T}$, or $A = U^\mathsf{T}U$ and $A^{-1} = U^{-1}U^{-\mathsf{T}}$;
- where a kernel reads the whole matrix (a product of two such factors, sums, right-hand sides of solves, LU, and uses outside the selected code: returns, calls, loops) the matrix is completed by `ulm_lamk.sycomplete`, which mirrors the stored triangle. It is created once, right after the definition of the matrix, and shared by all such uses. As every DPS op it works in place, and One-Shot Bufferize decides where: on the matrix itself if it may be overwritten (a temporary, or an `inout` parameter) and is not read afterwards, on a copy otherwise. Of two such factors of one product, the one that is complete already, else one that can be completed in place, is completed.

The interface comment names the triangle (`read only, lower triangle (symmetric)`). The tests fill the other triangle with NaN, so that any kernel reading it shows in the result.

The result of `syrk` is handled the same way. A Gram product $Z^\mathsf{T}Z$ that makes up a whole product and goes into a new matrix is computed by `syrk` (n^2k instead of $2n^2k$ FLOPs, and the DP prices it so); its result is passed on as stored in the lower triangle. The following Cholesky factorisation reads that triangle anyway; a return or a sum gets it completed (an n^2 copy instead of n^2k FLOPs). Gram products inside a larger chain, or reused by later products, stay `gemm`.

Costs. Every op implements `KernelCostOpInterface`, with FLOPs and bytes moved. Sizes come from static shapes. For dynamic dimensions (Section 8.1) the costs depend on run-time sizes, and kernel selection chooses at run time (next paragraph). Multi-versioning with a run-time dispatch on sizes is future work.

Scalars. `alpha` and `beta` are SSA values whose type must match the matrix element type (`TypesMatchWith`). Scales and accumulations folded into kernels need no special case.

7 Kernel selection

7.1 Problem

The input is a function of `ulm_lam` ops in normal form, with properties inferred. The output is a sequence of `ulm_lamk` ops (and `tensor.empty`) that computes the same values at minimal cost. Plain pattern rewriting cannot do this for three reasons:

- kernels match *subsequences* of n-ary products;
- choices interact. A Cholesky factorisation of M pays off only because both occurrences of M^{-1} reuse it;
- the best choice is visible only in the total cost.

7.2 The kernel catalogue

Kernels are declared, not hand-coded (Fig. 6). A record has five parts:

- a pattern over the normal form;
- constraints on properties, vector shapes and operand identity;
- the `ulm_lamk` op and its flags;
- the operand that is overwritten;
- a FLOP formula.

A custom TableGen backend (`ulm-lam-tblgen -gen-kernel-catalogue`) would generate the matchers, cost functions and builders. Today the file is checked for syntax with `mlir-tblgen --print-records`, which yields 31 records. Supporting another library, or a fused vendor kernel, means adding records. Linnea's kernel description plays the same role.

A `mul` pattern with k operands matches any k *consecutive* factors of a longer product. This restricted form of associative matching is all that products need, and it is linear in the product length. Commutative matching is needed only for sums. Sums are short in practice, so subsets can be enumerated. Full associative-commutative matching in the style of MatchPy is not required.

Factorisations are expression rewrites, as in Linnea. Cholesky replaces an spd leaf A by LL^{T}, with L lower triangular and non-singular. Normal form then turns A^{-1} into $L^{-\mathsf{T}}L^{-1}$. The ordinary `trsm`/`trsv` patterns pick up the solves, and no kernel needs to know about factorisations.

7.3 Inner solver: generalised matrix chain

For a single product $A_1 \cdots A_k$, whose leaves are decorated with ^{T} and $^{-1}$, the GMC algorithm [4] is a dynamic programme over intervals:

$$\mathrm{cost}(i,j) \;=\; \min_{i \le s < j} \; \min_{\kappa \in \mathscr{K}(T_{i,s},\, T_{s+1,j})} \Big[\mathrm{cost}(i,s) + \mathrm{cost}(s+1,j) + \mathrm{flops}_{\kappa} \Big],$$

where:

- $T_{i,s}$ is the temporary for the interval $A_i \cdots A_s$, with the properties inferred for it;
- $\mathscr{K}$ is the set of catalogue kernels that match the binary product of the two temporaries.

Because temporaries carry inferred properties, structure propagates through the chain. For example, $L^{-1}X$ stays full rank, so $(L^{-1}X)^{\mathsf{T}}(L^{-1}X)$ becomes `syrk` with an spd result. The cost is $O(k^3\,|\mathscr{K}|)$. GMC alone is a complete but myopic strategy (`strategy=gmc`).

7.4 Implemented: `strategy=gmc`

The implementation of `ulm-lam-select-kernels` (`lib/UlmLamPasses.cpp`) handles products, sums and inverses:

- **Inverses.** Every inverted matrix M is factorised once, right after its definition, chosen by its properties. A triangular M is used as it is; for an spd M, `potrf` gives $M = LL^{\mathsf{T}}$ and M^{-1} becomes the two solves $L^{-\mathsf{T}}L^{-1}$; any other M is factorised by `getrf`, and M^{-1} becomes an LU solve. The factorisation is shared by all products in which M^{-1} occurs, and it stays outside a loop in which M is invariant. An inverse is never formed explicitly; an inverse that is not a factor of a product is diagnosed.

- **Products.** Every product is parenthesised by the generalised matrix chain dynamic programme above. An interval is computed by a `gemm` (`gemv` for a vector), by a solve applied from the left to the rest (`trsm`, `trsv`, `getrs`, in place), by a triangular solve applied from the right (`trsm` with `side(right)`), or as $Z^\mathsf{T}Z$ when its left half is the transpose of its right half. The last case is Linnea's common subexpression up to transposition: $X^\mathsf{T}L^{-\mathsf{T}}L^{-1}X = (L^{-1}X)^\mathsf{T}(L^{-1}X)$. Transposition is always a kernel flag, never a copy. For example, $A\,B\,x$ with 1000×1000 matrices becomes $A(Bx)$, and $A^{-1}BC$ becomes $A^{-1}(BC)$ when C has fewer columns than B.
- **Reuse across products.** Every interval that a product computes as a plain product is remembered. In a later product, the same interval, or its transpose, costs nothing if the remembered value dominates the product. So the second product of GLS reuses $W = L^{-1}X$ from the first: $X^\mathsf{T}L^{-\mathsf{T}}L^{-1}y = W^\mathsf{T}(L^{-1}y)$, one triangular solve instead of two. This is a first step towards the sharing that Linnea's search finds (Section 7.5).
- **Sums.** $Y_1 + a_2Y_2 + \cdots + b_1P_1 + \ldots$ accumulates into the first plain term: `gescal` for its scale, `geaxpy` for every further plain term, and the last kernel of every product writes into the accumulator (`gemm` with $\beta = 1$; a solve through `geaxpy`). $X + 2A^\mathsf{T}B$ is therefore a single `gemm` with `trans_a(trans)`, $\alpha = 2$ and $\beta = 1$ that writes into X.
- **Diagnosed, not miscompiled:** explicit inverses, right solves with an LU factorisation, transposed accumulators, and transposes that are not kernel operands.

$Z^\mathsf{T}Z$ uses `syrk` where its result goes into a new matrix (Section 6). An inverse of a product of square, non-singular factors, $(F_1 \cdots F_k)^{-1}$, is inverted factor by factor, $F_k^{-1} \cdots F_1^{-1}$, when factorising the distinct factors costs less than forming the product and factorising it: $(RR)^{-1}$ with R spd is one Cholesky factorisation of R ($n^3/3$ instead of $2n^3 + n^3/3$, and $\kappa(R)$ instead of $\kappa(R)^2$). Products that read the same backwards when transposed, $F_1 \cdots F_k$ with $F_i^\mathsf{T} = F_{k+1-i}$, are symmetric (if the middle factor is), spsd, or spd: $A^\mathsf{T}A$, $A^\mathsf{T}PA$, but also RR and R^3.

The other passes are `ulm-lam-normalize` (flattening of products and sums, folding of double transposes and inverses, of transposes of symmetric matrices and of scaling by one), `ulm-lam-infer-properties` and `ulm-lam-strip-properties`. The dialects implement the property inference rules of Table 2 for products, sums, scaling, transposes and inverses, the relaxed type compatibility, the verifiers, FLOP and byte costs for every kernel, and bufferization of every `ulm_lamk` op in destination-passing style.

7.5 Outer solver: search over derivations

The default strategy is Linnea's search, ported.

- **State**: a hash-consed expression DAG of what remains to compute, plus the kernels emitted so far and their cost. Hash-consing makes CSE free. Nodes are canonicalised *modulo transposition*, so $X^\mathsf{T}L^{-\mathsf{T}}$ is found as $(L^{-1}X)^\mathsf{T}$ once $W = L^{-1}X$ exists.
- **Moves**: (i) apply a factorisation to a leaf (all its occurrences change together); (ii) compute a subexpression with GMC and replace it by a new leaf whose properties are inferred; (iii) apply a non-canonical algebraic rewrite (distribute, factor out).
- **Order**: best-first by cost so far plus a lower bound on the cost of the remainder. The search stops on a time budget (`budget-ms`). Like Linnea, it is anytime: the first complete derivation comes from a greedy descent, and better ones replace it.

The search works on its own data structure, not on IR. Cloning functions for every state would be wasteful. A search state is also not a valid program: partly selected expressions mix factors and pending products. The DAG reuses the C++ property functions of the ops, so inference in the search and in the IR cannot disagree. Only the winning derivation is materialised as `ulm_lamk` ops. A debug option dumps the explored derivation graph.

7.6 Alternative: equality saturation

`strategy=egraph` is the experimental route. It translates the `ulm_lam` function into an e-graph, through the `eqsat` dialect [12] or DialEgg/egglog [13]. It saturates with the algebraic identities and the catalogue rules, then extracts the cheapest term. The idea is attractive, because the moves of Section 7.5 become plain rewrite rules. There are three difficulties:

- properties are per e-class facts. egglog lattices with `:merge` can represent them, but rewrites become conditional on analyses;
- extraction with sharing is NP-hard, and integer programming is the usual approach. This matches the NP-completeness of the LAMP;
- associativity makes the e-graphs of products blow up. GMC as a custom extractor for chain e-classes is one way out.

A comparison of both strategies on the LAMP benchmark [1] would itself be a contribution.

7.7 Soundness and numerics

Kernel selection must not change the result beyond rounding. Rewrites with side conditions are checked against inferred properties:

- $(AB)^{-1} = B^{-1}A^{-1}$ only for square non-singular factors;
- $A^{\mathsf{T}} = A$ only for symmetric A.

Explicit inverses are never formed unless the inverse itself is an output. Factorisations are chosen by structure: Cholesky for spd, LU with pivoting otherwise, and QR (orthogonal Q times upper triangular R) where requested. Every factorisation reports failure through the `info` of the function (Section 8.1), so a violated property assumption is reported to the caller instead of producing garbage.

8 Bufferization and lowering

Buffers. After `ulm-lam-strip-properties`, the IR consists of `ulm_lamk` ops on plain tensors. `ulm-lam-results-to-out-params` first makes every matrix result a by-reference parameter (Section 8.1). The pipeline then runs upstream passes in four steps:

1. `one-shot-bufferize{bufferize-function-boundaries}`;
2. `buffer-results-to-out-params{hoist-static-allocs}`;
3. `buffer-loop-hoisting`, which moves allocations of loop-invariant size out of loops (Section 8.1);
4. `buffer-deallocation-pipeline`.

`ulm-lam-results-to-out-params` marks the matrices passed by value `bufferization.writable = false`. The analysis therefore copies an input only when a kernel overwrites it. In the example these are M (`potrf`), X (`trsm`) and y (`trsv`). Intermediate results that end in the output are computed directly in the result buffer. That is Linnea's memory optimisation, here obtained from upstream bufferization.

8.1 Lowering to ulmBLAS

`convert-ulm-lamk-to-ulmblas` (`lib/ConvertUlmLamkToUlmBLAS.cpp`) lowers the bufferized program to EmitC, MLIR's dialect for emitting C and C++. Every kernel becomes one call of the C++ interface of ulmBLAS [16], and `mlir-translate --mlir-to-cpp` prints C++17. Loops, arithmetic and functions use the upstream `{scf,arith,func}-to-emitc` patterns with the type converter of the pass.

Why ulmBLAS and not CBLAS/LAPACK. ulmBLAS is header only, its element type is a template parameter, and every matrix is a pointer with two strides (`incRow`, `incCol`), for BLAS and LAPACK routines alike. Strides are what an MLIR `memref` carries, so the lowering is direct:

- a transposed operand is the same pointer with swapped strides, so `trans` flags disappear, and the transpose of a lower triangular matrix is an upper triangular one (`trusv` on L with strides $(1, n)$ solves with L^T);
- a triangular solve from the right is a solve from the left on transposed views: $B\,\mathrm{op}(T)^{-1} = (\mathrm{op}(T)^{-\mathsf{T}} B^\mathsf{T})^\mathsf{T}$, so `side` disappears too;
- row-major versus column-major is not an issue: LAPACK routines such as `potrf`, `getrf` and `getrs` take strides as well. With CLAPACK, row-major data needs tricks (a row-major lower triangle is the column-major upper one; `getrf` factorises A^T) and `getrs` with several right-hand sides cannot be used at all;
- factorisations return `info`, which the generated function turns into its own `info` (Section 8.1).

A second lowering: CBLAS and LAPACK. ulm.lam does two things at once: it maps formulas onto BLAS and LAPACK routines (the LAMP), and it runs them on ulmBLAS. To measure the first independently of the second, the option `backend=cblas` of `convert-ulm-lamk-to-ulmblas` (`BACKEND=cblas` for `scripts/ulm-lam-compile.sh`, `lam.compile(..., backend="cblas")` in Python) emits the same calls with the namespace `ulm_lam_cblas` instead of `ulmBLAS`. The header `include/ulm_lam/cblas_backend.h` implements that interface with CBLAS and the Fortran LAPACK symbols, which MKL (Intel Math Kernel Library), OpenBLAS and Accelerate all provide. It works in column-major order and uses the tricks above instead of copies: a row-major operand is the transposed column-major one (a trans flag, a flipped triangle, or the transposed problem $C^\mathsf{T} = B^\mathsf{T} A^\mathsf{T}$), `potrf` factorises the flipped triangle, and `getrf` factorises A^T, which `getrs`, called with the same strides, takes into account. Only a right-hand side of `getrs` and the B of `symm` whose orientation differs from that of C are copied ($O(nm)$). The kernel sequence is identical, so the time of the generated code on MKL compared with the hand-written MKL version measures the mapping, and the same code on ulmBLAS compared with it on MKL measures ulmBLAS (Section 13.1). All 42 Python tests pass with both backends.

The ulm.lam ABI.

- A static row-major `memref` is a pointer (`double *`, `float *`, pivots `int32_t *`) with strides (columns, 1); `index` is `size_t`. So `foo(index, memref<500x500xf64>)` is `void foo(size_t, double *)`.
- Matrices are passed by reference. A function with a matrix output receives a pointer to it; the caller allocates and frees the matrix and owns it. No function returns a matrix, so ownership never moves from callee to caller, and an external function cannot leak. `ulm-lam-results-to-out-params` and `buffer-results-to-out-params` establish the convention; `convert-ulm-lamk-to-ulmblas` rejects a function that violates it.
- Every matrix parameter has an intent, `in`, `inout` or `out`, declared in the parameter list (see below).
- An external function that updates a matrix receives it by reference and changes it in place: a declaration whose result is marked `{ulm_lam.inout = k}` becomes a `ulm_lamk.call` (Section 10.1).
- A function that needs memory for temporaries gets it from the caller, in the style of LAPACK's `lwork` (Section 8.1).
- Functions are `extern "C"`: C code calls the generated C++, and the generated C++ calls C functions.

Parameter intents. LAPACK routines commonly overwrite the matrices they receive; the caller learns it from the documentation ("on exit, A is destroyed"). In ulm.lam the parameter list says it:

- `{ulm_lam.in}` (the default for matrix arguments): read only. If a kernel would overwrite the matrix, the function works on a copy, which lives in its workspace.
- `{ulm_lam.inout}`: the function may overwrite the matrix and use it as workspace. If a result is marked `{ulm_lam.inout = k}`, argument k holds that result on exit (the loop example updates X this way); otherwise its contents on exit are unspecified.
- `{ulm_lam.out}` (the default for results): an out parameter; the caller provides the buffer, the function writes it and also computes in it (`gemv` and the triangular solves of GLS work in the caller's b).

`ulm-lam-results-to-out-params` turns the intents into bufferization attributes (`in` arguments are not writable, `inout` arguments are), so One-Shot Bufferize decides copies and in-place updates accordingly. `examples/gls/gls_inout.mlir` is GLS with M and y declared `inout`: the Cholesky factorisation of M and the solve with y work on the caller's matrices, only X (`in`) is copied for the in-place solve $W = L^{-1}X$, and the workspace shrinks from 35.5 MB to 3.5 MB. The result is bit-identical to that of `gls`.

Because the caller has to know the intents, the generated code states them: every public function is preceded by a comment generated from its parameter attributes,

```
/* gls_inout: matrices by reference (row-major), owned by the caller
 *   v1  double[2000][200]   in     read only
 *   v2  double[2000][2000]  inout  may be overwritten (unspecified on exit)
 *   v3  double[2000][1]     inout  may be overwritten (unspecified on exit)
 *   v4  double[200][1]      out    written
 *   v5  int8_t *            work   workspace (bytes)
 *   v6  int64_t *           info   -1: query, 0: allocate, >0: size of work
 */
```

Memory: `memref.alloc`, `dealloc` and `copy` become `aligned_alloc` (size rounded up to the alignment, as C++17 requires), `free` and `ulm_lam_copy`, a `memcpy` that copies large arrays in parallel when compiled with OpenMP. On a machine with several NUMA (Non-Uniform Memory Access) domains the first write decides where a page of a new buffer lies: a serial copy into a workspace the function allocated itself would put all of it on one domain, and the kernels that then use it with all threads pay for that (Section 13.1). Upstream `convert-memref-to-emitc` is not used: it types arguments as C arrays (`double[500][500]`) but allocations as pointers, which leaves unresolved casts.

Dynamic sizes. Matrix sizes can be run-time parameters. The argument (and result) attribute `ulm_lam.dims` ties every dynamic dimension of a matrix to an index parameter of its function (argument positions, one per `?`):

```
func.func @gls(%n: index, %m: index,
               %X: tensor<?x?xf64, #fr> {ulm_lam.dims = [0, 1]},
               %M: tensor<?x?xf64, #spd> {ulm_lam.dims = [0, 0]},
               %y: tensor<?x1xf64> {ulm_lam.dims = [0]})
    -> (tensor<?x1xf64> {ulm_lam.dims = [1]})
```

The generated function takes the sizes as `size_t` parameters, `gls(size_t v1, size_t v2, double *v3, ...)`, and the interface comment says `double[v1][v2]`. `ulm_lam::dimOf` traces the size of a dimension through the ops of the pipeline to a parameter, an allocation, or a call result (whose callee names the size by `ulm_lam.dims` on its result); kernel selection, the workspace layout and the lowering use it. The results of a public function become out parameters before bufferization, and with `eliminate-empty-tensors` the kernels write into the caller's buffer directly (`buffer-results-to-out-params` can only do that for static shapes). The workspace query is computed from the sizes and returns what exactly the evaluation

chosen by the compiler needs. All ulmBLAS calls name their index type, `ulmBLAS::gemm<std::ptrdiff_t>`, so that literals and `size_t` parameters convert to it. `examples/dynamic` checks one compiled GLS for six sizes, from 1×1 to 2000×200.

Plans chosen at run time. Which evaluation is best can depend on the sizes: for $D = ABC$ with $A \in \mathbb{R}^{n \times k}$, $B \in \mathbb{R}^{k \times m}$, $C \in \mathbb{R}^{m \times p}$, $(AB)C$ costs $2nkm + 2nmp$ and $A(BC)$ costs $2kmp + 2nkp$. Kernel selection runs the matrix-chain DP for every assignment of $\{10, 1000\}$ to the (at most four) distinct dynamic sizes of a product; if the optimal plans differ, the generated code computes the cost of each from the actual sizes (index arithmetic, as the DP does) and runs the cheapest (an `scf.if` chain):

```
if (((v1 * v2) * v3) * 2 + ((v1 * v3) * v4) * 2 <=
    ((v2 * v3) * v4) * 2 + ((v1 * v2) * v4) * 2) {
    ulmBLAS::gemm<std::ptrdiff_t>(v1, v3, v2, ..., v5, ..., v6, ..., T, ...);  // T = A B
    ulmBLAS::gemm<std::ptrdiff_t>(v1, v4, v3, ..., T, ..., v7, ..., v8, ...);  // D = T C
} else {
    ulmBLAS::gemm<std::ptrdiff_t>(v2, v4, v3, ..., v6, ..., v7, ..., T, ...);  // T = B C
    ulmBLAS::gemm<std::ptrdiff_t>(v1, v4, v2, ..., v5, ..., T, ..., v8, ...);  // D = A T
}
```

Every plan writes the product into the caller's buffer. The workspace parts of the plans overlap, and the workspace query evaluates the same condition, so it returns the size of exactly the plan that runs: `v16 ? round64(8 * v1 * v3) : round64(8 * v2 * v4)`. Within a plan no subproduct is reused by later products (it would not dominate them), and no `syrk` result is passed on as one triangle. `examples/dynamic/driver_chain.cc` checks sizes for which either plan wins.

Workspaces. A generated function does not allocate memory behind the caller's back. `ulm-lam-workspace` (`lib/UlmLamWorkspace.cpp`) runs after bufferization. A public function with temporaries gets two trailing parameters:

```
void gls(double *X, double *M, double *y, double *b,
         int8_t *work, int64_t *workspace_info);
```

- `*workspace_info == -1`: a query. The function sets `*workspace_info` to the required size in bytes and returns without computing anything.
- `*workspace_info == 0`: the function allocates one block of the required size, uses it, and frees it before it returns.
- `*workspace_info > 0`: the size of `work`. It must be at least the required size; if it is too small, the function returns $info = -i$, where i is the position of `workspace_info` (as LAPACK reports an illegal `lwork`), and computes nothing. Otherwise all temporaries live in `work`.

The sizes are in bytes, because the temporaries mix element types (matrices and `int32_t` pivots). All temporaries, including those of loops, become slots of the one block at 64-byte aligned offsets (computed at compile time, or from the sizes for dynamic dimensions); in the C++ each is `(double *)(work + offset)`. The query returns the size that exactly the evaluation chosen by the compiler needs; there is no separate minimal size as in LAPACK. Functions without temporaries keep their plain signature.

Accumulation. As a LAPACK routine that calls other routines, a function that calls functions with a workspace answers a query with its own requirement plus those of its callees. `ulm-lam-workspace` decides transitively which functions take a workspace: those with temporaries, and those that call a function with a workspace. An external function declares that it follows the convention with the attribute `{ulm_lam.workspace}`; its last two parameters are then `work` and `workspace_info`. At entry, a function queries each such call (`*workspace_info = -1`; the matrix arguments are dummies, which a query does not touch; scalars that are not available at entry, such as a loop index, are 0) and restores

its own `*workspace_info`. Its block is laid out as [own temporaries | callee 1 | callee 2 | ...], each part 64-byte aligned, and every call gets its part and its size. Afterwards the function restores `*workspace_info`, as LAPACK leaves `lwork` unchanged. So the requirement of an external function with a workspace may depend only on its matrix sizes, which are static in ulm.lam, and on scalar arguments that are available at entry. Recursion between functions with a workspace is rejected.

`examples/compose/compose.mlir` has a generated function `estimate_all` that calls the generated `gls` and the external `postprocess` in a loop (Fig. 7). Its query returns 35 555 200 bytes: its own 17 600 (y_i and b_i) plus the 35 536 000 of `gls` plus the 1 600 of `postprocess`. `examples/compose/driver.cc` checks the accumulation, that every call of `postprocess` gets a large enough part of the caller's workspace, that the estimates equal separate calls of `gls`, both with a given workspace and with `*workspace_info = 0`, that `*workspace_info` is unchanged after the call, that a workspace one byte too small gives $info = -5$ and computes nothing, and that a failing `gls` (an indefinite M) makes `estimate_all` return $info = 1$ (its call of `gls`) at once.

GLS needs 35.5 MB (copies of M, X, y and the matrix S); the driver queries the size once and passes the same workspace to every call. `examples/cases/driver.cc` checks all four cases: the query computes nothing, a given workspace gives the same result, and a workspace that is one byte too small gives $info = -5$ and computes nothing; and that an indefinite matrix assumed spd gives $info = 1$ (its Cholesky factorisation) and a singular one $info = 1$ (its LU factorisation), without aborting.

Info. Numerically, a matrix can fail a property that holds mathematically: a matrix that is positive definite in exact arithmetic need not be so in floating point, and the factorisation then breaks down. LAPACK leaves the decision to the caller: its routines return `info`, and the caller either chooses another method or returns an `info` of its own. Every function that ulm.lam generates does the same:

```
extern "C" int64_t gls(size_t n, size_t m, double *X, double *M, double *y,
                       double *b, int8_t *work, int64_t *workspace_info);
/* returns info:
 *    0  success
 *    1  Cholesky factorisation of v4: not positive definite
 *    2  Cholesky factorisation of a computed matrix: not positive definite
 *   -8  v8: workspace too small
 * (info != 0: the outputs are unspecified) */
```

$info = k > 0$ says that check k failed: a factorisation (`potrf`: not positive definite; `getrf`: singular) or a call of another ulm.lam function that returned a non-zero `info`; $info = -i$ that argument i is illegal, which is the case for a workspace that is too small. The checks are numbered in the order of the function body, and the interface comment lists them. `ulm-lam-info` implements this on buffers, after `ulm-lam-workspace`: the factorisation kernels return LAPACK's `info`, and after every check the rest of its block runs only if the check passed (an `scf.if` whose result is the status of the block). A loop carries the status: once an iteration has failed, the remaining ones do nothing, and the function returns the code of the failure. Releasing a workspace that the function allocated itself runs also after a failure. The guards cost nothing measurable: the Kalman loop of Section 10 takes the same time as without them.

In Python, a non-zero `info` raises `lam.InfoError`, whose message is the line of the interface comment, e.g. `gls: info 1: Cholesky factorisation of v4: not positive definite`. A fallback, such as an LU factorisation where a Cholesky factorisation fails, will use the same mechanism as choosing a plan by the sizes at run time: several plans for one formula in the generated code.

What ulm.lam needs from ulmBLAS next. ulm.lam also shows how missing ulmBLAS routines should look: in the style of the existing ones, with strides for every matrix, a template element type, `bool` flags only where strides cannot express the choice, and `info` as the return value of factorisations.

- *The triangle of a product*: `sylrk` is used for $Z^\mathsf{T}Z$ (Section 6); `gemmtr` (in ulmBLAS `master`) would halve the second product of HPH^T and $A^\mathsf{T}PA$, whose results are symmetric although the factors

are not transposes of each other.

- *QR factorisation* for least squares without normal equations, and for full-rank inverses that are not spd:

```
template <typename IndexType, typename T>
    IndexType
    geqrf(IndexType m, IndexType n, T *A, IndexType incRowA,
          IndexType incColA, T *tau, IndexType incTau);

//  C := op(Q) C (left) or C op(Q) (right), Q from geqrf
template <typename IndexType, typename T>
    void
    ormqr(bool left, bool trans, IndexType m, IndexType n, IndexType k,
          const T *A, IndexType incRowA, IndexType incColA,
          const T *tau, IndexType incTau,
          T *C, IndexType incRowC, IndexType incColC);
```

- *Symmetric indefinite factorisation* $PAP^{\mathsf{T}} = LDL^{\mathsf{T}}$ (Bunch–Kaufman) for inverses of symmetric matrices that are not known to be definite, with `sytrf` and `sytrs` shaped like `getrf` and `getrs` (pivots 0-based, increment `incPiv`).

The workspace convention of ulm.lam (Section 8.1) is the one of LAPACK-level routines: BLAS routines never need workspace, LAPACK routines such as `getri` do. A routine that calls other routines with a workspace answers a query with the accumulated requirement of its own temporaries and of its callees.

Everything the implemented kernel selection emits is already in ulmBLAS: `gemm`, `gemv`, `trlsm`/`trusm`, `trlsv`/`trusv`, `geaxpy`, `gescal`, `potrf`, `getrf` and `getrs`.

Reference lowering. `convert-ulm-lamk-to-linalg` (planned) lowers kernels without a library: `gemm` to `linalg.matmul`, triangular solves and factorisations to `scf` loops. Running it next to `convert-ulm-lamk-to-ulmblas` gives differential tests.

Loops and temporaries. Consider matrix operations inside a loop:

$$\textbf{for } i = 0, \ldots, n-1 : \quad B = \mathrm{foo}(i), \quad X \mathrel{+}= ABC.$$

Suppose kernel selection chose $(AB)C$. The loop body is then a `gemm` into a temporary $T = AB$ with $\beta = 0$, followed by a `gemm` $X = TC + X$ with $\beta = 1$ into the loop-carried X. `examples/loop/` contains this loop, written with `ulm_lam` ops as a frontend would emit it (Fig. 8). `ulm-lam-select-kernels` turns it into exactly these two gemms (Fig. 9). With upstream passes alone, the result then bufferizes badly:

- One-Shot Bufferize allocates T *inside* the loop, and the deallocation pass adds run-time ownership bookkeeping to every iteration;
- foo allocates B itself, so B is freed in every iteration. `buffer-results-to-out-params` rewrites only functions with a body, and `foo` is external;
- X is returned by value. Even with `modify-public-functions`, the result becomes a separate out parameter, and the updated X is copied into it after the loop. `drop-equivalent-buffer-results` skips public functions.

Matrices by reference. `ulm-lam-results-to-out-params` fixes the calling convention before bufferization. It is implemented in `lib/UlmLamResultsToOutParams.cpp` and does two things:

- $X = \mathrm{foo}(i)$ is rewritten into $\mathrm{foo}(i, X)$ with X passed by reference. The external declaration gets one `memref` parameter per matrix result and returns nothing. Every call becomes a `memref.alloc`,

the call `foo(i, X)`, and a `bufferization.to_tensor restrict writable` that makes the written buffer available to the tensor program. In C, this is the signature `void foo(size_t i, double *X)` (EmitC maps `index` to `size_t`);

- a result marked `{ulm_lam.inout = 2}` is the updated value of argument 2. The argument becomes a `memref`, it is read once with `to_tensor`, the final value is written back with `bufferization.materialize_in_destination`, and the result is dropped. X is updated in place through the reference. Such functions must not be called inside the module; the pass rejects that case with a diagnostic.

After this pass, the upstream passes `one-shot-bufferize`, `buffer-loop-hoisting` and `buffer-deallocation-pipeline` place every buffer (Figs. 8 and 10):

- B and T get their buffers once, before the loop: after `ulm-lam-workspace` they are two slots of the caller's workspace;
- X is updated in place;
- the loop body contains only the four calls, with no allocation, copy or deallocation.

`examples/loop/check.sh` (`make loop`) runs the pipeline and checks these properties.

The loop in C++. The loop example is lowered from `ulm_lam` ops to C++ calling ulmBLAS by passes alone (`scripts/ulm-lam-compile.sh`); there is no hand-written stage. Figure 11 shows the result: two calls of `ulmBLAS::gemm` per iteration, the second one with $\beta = 1$ into X, and `foo` writing B into memory that `accumulate` allocated and owns. `examples/loop/run.sh` (`make loop`) compiles it with `c++ -std=c++17 -Wall -Wextra -pedantic` (no warnings in the generated code), links it with a C driver that defines `foo`, and compares the result with naive C loops. For $n = 500$ and 4 iterations (one run), it takes 13.2 ms against 118 ms for the naive loops; the relative difference is $1.3 \cdot 10^{-15}$. The loop needs a 4 MB workspace for B and T.

The full pipeline. `scripts/ulm-lam-compile.sh` runs it in three steps and keeps every stage as a file:

```
mlir-opt --load-dialect-plugin=UlmLamPasses.dylib \
         --load-pass-plugin=UlmLamPasses.dylib --pass-pipeline='builtin.module(
    func.func(ulm-lam-normalize, ulm-lam-infer-properties, ulm-lam-select-kernels),
    ulm-lam-strip-properties, canonicalize,
    ulm-lam-results-to-out-params, eliminate-empty-tensors,
    one-shot-bufferize{bufferize-function-boundaries
                       function-boundary-type-conversion=identity-layout-map},
    buffer-results-to-out-params{hoist-static-allocs modify-public-functions},
    func.func(buffer-loop-hoisting), buffer-deallocation-pipeline,
    canonicalize, ulm-lam-workspace, canonicalize, cse,
    ulm-lam-info, canonicalize,
    convert-ulm-lamk-to-ulmblas, reconcile-unrealized-casts, form-expressions)'
mlir-translate --mlir-to-cpp gls.emitc.mlir -o gls.cpp
clang-format -i gls.cpp
c++ -std=c++17 -O3 -mcpu=native -isystem $ULMBLAS_DIR -c gls.cpp
```

`ARCHFLAGS` (default `-mcpu=native`) selects the micro kernels of ulmBLAS, e.g. `-mcpu=apple-m4` or `-mavx -mfma`.

9 Worked example: generalised least squares

The example is Eq. (1) with $n = 2000$ observations and $m = 200$ parameters. The five files in `examples/gls/` show the IR after each step.

Stage 1: frontend output. The frontend builds binary products left to right and shares nothing. Builders infer local properties: M^{-1} is spd, X^{T} is full rank. The binary product $(X^{\mathsf{T}}M^{-1})X$ is not yet known to be spd.

```
// Stage 1: frontend output.
//
//   b = (X^T M^-1 X)^-1 X^T M^-1 y,   M spd, X full rank
//
// The frontend builds binary operations left to right and does not share
// subexpressions.  Result properties are inferred locally by the op
// builders (InferTypeOpInterface): M^-1 is spd, X^T is full rank -- but the
// binary product (X^T M^-1) X is not recognised as spd yet.

#fr  = #ulm_lam.props<full_rank>
#spd = #ulm_lam.props<spd>

!X  = tensor<2000x200xf64, #fr>
!Xt = tensor<200x2000xf64, #fr>
!M  = tensor<2000x2000xf64, #spd>
!y  = tensor<2000x1xf64>
!T  = tensor<200x2000xf64>
!G  = tensor<200x200xf64>
!b  = tensor<200x1xf64>

func.func @gls(%X: !X, %M: !M, %y: !y) -> !b {
  %0 = ulm_lam.transpose %X : !X -> !Xt
  %1 = ulm_lam.inverse %M : !M -> !M
  %2 = ulm_lam.mul %0, %1 : !Xt, !M -> !T
  %3 = ulm_lam.mul %2, %X : !T, !X -> !G
  %4 = ulm_lam.inverse %3 : !G -> !G
  %5 = ulm_lam.transpose %X : !X -> !Xt
  %6 = ulm_lam.mul %4, %5 : !G, !Xt -> !T
  %7 = ulm_lam.inverse %M : !M -> !M
  %8 = ulm_lam.mul %6, %7 : !T, !M -> !T
  %9 = ulm_lam.mul %8, %y : !T, !y -> !b
  return %9 : !b
}
```

Stage 2: normal form. Products are flattened, and the duplicated X^{T} and M^{-1} are shared. $X^{\mathsf{T}}M^{-1}X$ is now recognised as spd by the rule $A^{\mathsf{T}}PA$.

```
// Stage 2: after ulm-lam-normalize, cse, ulm-lam-infer-properties.
//
// Products are flattened (no parenthesisation is fixed any more), the
// duplicated X^T and M^-1 are shared, and the flattened product
// X^T M^-1 X is now recognised as spd:
//   A^T P A  with P spd and A full (column) rank  =>  spd.

#fr  = #ulm_lam.props<full_rank>
#spd = #ulm_lam.props<spd>

!X  = tensor<2000x200xf64, #fr>
!Xt = tensor<200x2000xf64, #fr>
!M  = tensor<2000x2000xf64, #spd>
!y  = tensor<2000x1xf64>
!S  = tensor<200x200xf64, #spd>
!b  = tensor<200x1xf64>

func.func @gls(%X: !X, %M: !M, %y: !y) -> !b {
  %Xt = ulm_lam.transpose %X : !X -> !Xt
  %Mi = ulm_lam.inverse %M : !M -> !M
  %S  = ulm_lam.mul %Xt, %Mi, %X : !Xt, !M, !X -> !S
  %Si = ulm_lam.inverse %S : !S -> !S
```

```
  %b  = ulm_lam.mul %Si, %Xt, %Mi, %y : !S, !Xt, !M, !y -> !b
  return %b : !b
}
```

Derivation. Table 6 shows Linnea's derivation, which stages 3 and 4 spell out by hand, next to the one `ulm-lam-select-kernels` finds from stage 2 by itself. Both use Cholesky on M, which turns both occurrences of M^{-1} into $L^{-\mathsf{T}}L^{-1}$, compute $W = L^{-1}X$ once, form $S = W^{\mathsf{T}}W$ from it (the common subexpression up to transposition), and reuse it for the right-hand side, $W^{\mathsf{T}}z$ with $z = L^{-1}y$. S is spd, so the second Cholesky factorisation turns S^{-1} into solves. The kernel sequences are the same, including `syrk` for $W^{\mathsf{T}}W$ (Section 6).

```
// Stage 3: after ulm-lam-select-kernels and ulm-lam-strip-properties.
//
// The derivation found by the search (FLOPs for n = 2000, m = 200):
//
//   M = L L^T            potrf   n^3/3     2.67e9
//   W = L^-1 X           trsm    n^2 m     8.00e8
//   z = L^-1 y           trsv    n^2       4.00e6
//   S = W^T W            syrk    m^2 n     8.00e7   (spd: W full rank)
//   S = K K^T            potrf   m^3/3     2.67e6
//   c = W^T z            gemv    2 n m     8.00e5
//   b = K^-T (K^-1 c)    2x trsv 2 m^2     8.00e4
//                                total     3.55e9
//
// Kernel results have BLAS semantics: potrf/syrk define the lower
// triangle only; the strict upper triangle keeps the init's values
// (undefined for tensor.empty).  Every consumer reads the lower triangle.

!X = tensor<2000x200xf64>
!M = tensor<2000x2000xf64>
!y = tensor<2000x1xf64>
!S = tensor<200x200xf64>
!b = tensor<200x1xf64>

func.func @gls(%X: !X, %M: !M, %y: !y) -> !b {
  %one  = arith.constant 1.0 : f64
  %zero = arith.constant 0.0 : f64

  %L, %infoL = ulm_lamk.potrf uplo(lower) outs(%M : !M) -> !M
  %W = ulm_lamk.trsm side(left) uplo(lower) trans(none) diag(non_unit)
         alpha(%one) ins(%L : !M) outs(%X : !X) -> !X
  %z = ulm_lamk.trsv uplo(lower) trans(none) diag(non_unit)
         ins(%L : !M) outs(%y : !y) -> !y

  %e0 = tensor.empty() : !S
  %S  = ulm_lamk.syrk uplo(lower) trans(trans) alpha(%one) beta(%zero)
          ins(%W : !X) outs(%e0 : !S) -> !S
  %K, %infoK = ulm_lamk.potrf uplo(lower) outs(%S : !S) -> !S

  %e1 = tensor.empty() : !b
  %c  = ulm_lamk.gemv trans(trans) alpha(%one) beta(%zero)
          ins(%W, %z : !X, !y) outs(%e1 : !b) -> !b
  %d  = ulm_lamk.trsv uplo(lower) trans(none) diag(non_unit)
          ins(%K : !S) outs(%c : !b) -> !b
  %b  = ulm_lamk.trsv uplo(lower) trans(trans) diag(non_unit)
          ins(%K : !S) outs(%d : !b) -> !b
  return %b : !b
}
```

Stage 4: bufferized (design). Bufferization inserts exactly three copies, one for each overwritten input. c, d and b share the result buffer.

```
// Stage 4: after one-shot-bufferize{bufferize-function-boundaries} and
// buffer-results-to-out-params{hoist-static-allocs}.
//
// The arguments are marked bufferization.writable = false by ulm-lam-compile
// (callers' inputs are not clobbered unless they opt in), so the analysis
// inserts exactly the copies Linnea would: M (overwritten by potrf),
// X (overwritten by trsm) and y (overwritten by trsv).  S needs a fresh
// buffer.  c, d and b are written straight into the result buffer, because
// every trsv updates its outs operand in place.

!X = memref<2000x200xf64>
!M = memref<2000x2000xf64>
!y = memref<2000x1xf64>
!S = memref<200x200xf64>
!b = memref<200x1xf64>

func.func @gls(%X: !X, %M: !M, %y: !y, %b: !b) {
  %one  = arith.constant 1.0 : f64
  %zero = arith.constant 0.0 : f64

  %L = memref.alloc() {alignment = 64 : i64} : !M
  memref.copy %M, %L : !M to !M
  %infoL = ulm_lamk.potrf uplo(lower) outs(%L : !M)

  %W = memref.alloc() {alignment = 64 : i64} : !X
  memref.copy %X, %W : !X to !X
  ulm_lamk.trsm side(left) uplo(lower) trans(none) diag(non_unit)
    alpha(%one) ins(%L : !M) outs(%W : !X)

  %z = memref.alloc() {alignment = 64 : i64} : !y
  memref.copy %y, %z : !y to !y
  ulm_lamk.trsv uplo(lower) trans(none) diag(non_unit) ins(%L : !M) outs(%z : !y)

  %S = memref.alloc() {alignment = 64 : i64} : !S
  ulm_lamk.syrk uplo(lower) trans(trans) alpha(%one) beta(%zero)
    ins(%W : !X) outs(%S : !S)
  %infoS = ulm_lamk.potrf uplo(lower) outs(%S : !S)

  ulm_lamk.gemv trans(trans) alpha(%one) beta(%zero)
    ins(%W, %z : !X, !y) outs(%b : !b)
  ulm_lamk.trsv uplo(lower) trans(none) diag(non_unit) ins(%S : !S) outs(%b : !b)
  ulm_lamk.trsv uplo(lower) trans(trans) diag(non_unit) ins(%S : !S) outs(%b : !b)

  memref.dealloc %S : !S
  memref.dealloc %z : !y
  memref.dealloc %W : !X
  memref.dealloc %L : !M
  return
}
```

Stage 5: generated. `examples/gls/run.sh` compiles stage 2 with `scripts/ulm-lam-compile.sh`. Figure 12 shows the kernels that `ulm-lam-select-kernels` chooses, Fig. 13 the C++ after bufferization and `convert-ulm-lamk-to-ulmblas`. Bufferization copies exactly M, X and y, the inputs that kernels overwrite, and b is written into the caller's buffer. In the C++, the solve with K^{T} is `ulmBLAS::trusv` on the same factor K with strides $(1, 200)$, and W^{T} is the same pointer as W with strides $(1, 200)$, in the `gemm` for $W^{\mathsf{T}}W$ and in the `gemv` for $W^{\mathsf{T}}z$.

```
gls(double *v1, double *v2, double *v3, double *v4, int8_t *v5, int64_t *v6)
{
```

```cpp
    int64_t v7 = 35536000;
    double v8 = 1.0000000000000000e+00;
    double v9 = 0.0e+00;
    int64_t v10 = 0;
    size_t v11 = 0;
    int64_t v12 = v6[v11];
    int64_t v13;
    if (v12 == -1) {
        v6[v11] = v7;
        v13 = v10;
    } else {
        int64_t v14;
        if (v12 > v10 & v12 < v7) {
            v14 = -6;
        } else {
            bool v15 = v12 == v10;
            int8_t *v16;
            if (v15) {
                void *v17 = aligned_alloc(64, 35536000);
                v16 = (int8_t *)v17;
            } else {
                v16 = v5;
            }
            int8_t *v18 = v16;
            int64_t v19 = (int64_t)v11;
            double *v20 = (double *)(v18 + v19);
            ulm_lam_copy(v20, v2, 32000000);
            int64_t v21 =
                ulmBLAS::potrf<std::ptrdiff_t>(true, 2000, v20, 2000, 1);
            bool v22 = (bool)(v21 != v10);
            uint64_t v23 = (uint64_t)v22;
            int64_t v24 = (int64_t)v23;
            int64_t v25;
            if (v24 == v10) {
                int64_t v26 = (int64_t)32000000;
                double *v27 = (double *)(v18 + v26);
                int64_t v28 = (int64_t)32320000;
                double *v29 = (double *)(v18 + v28);
                ulm_lam_copy(v29, v1, 3200000);
                (void)ulmBLAS::trtrs<std::ptrdiff_t>(true, false, false, false,
                                                     2000, 200, v20, 2000, 1,
                                                     v29, 200, 1);
                ulmBLAS::sylrk<std::ptrdiff_t>(200, 2000, v8, v29, 1, 200, v9,
                                               v27, 200, 1);
                int64_t v30 =
                    ulmBLAS::potrf<std::ptrdiff_t>(true, 200, v27, 200, 1);
                int64_t v31 = v30 != v10 ? 2 : v10;
                if (v31 == v10) {
                    int64_t v32 = (int64_t)35520000;
                    double *v33 = (double *)(v18 + v32);
                    ulm_lam_copy(v33, v3, 16000);
                    ulmBLAS::trlsv<std::ptrdiff_t>(2000, false, v20, 2000, 1,
                                                   v33, 1);
                    ulmBLAS::gemv<std::ptrdiff_t>(200, 2000, v8, v29, 1, 200,
                                                  v33, 1, v9, v4, 1);
                    ulmBLAS::trlsv<std::ptrdiff_t>(200, false, v27, 200, 1, v4,
                                                   1);
                    ulmBLAS::trusv<std::ptrdiff_t>(200, false, v27, 1, 200, v4,
                                                   1);
                }
                v25 = v31;
            } else {
                v25 = v24;
```

```
            }
            int64_t v34 = v25;
            if (v15) {
                free(v18);
            }
            v14 = v34;
        }
        v13 = v14;
    }
    return v13;
}
```

Figure 13: GLS in C++ calling ulmBLAS, generated from stage 2: the function `extern "C" int64_t gls(...)`, without the `#include` lines in front of it. The outer `if`s implement the workspace convention (Section 8.1), the inner ones the `info` of Section 8.1.

Results. `driver.cc` uses ulmBLAS for everything:

- it builds a random X, an spd $M = AA^{\mathsf{T}} + nI$ and a random y;
- it calls the generated `gls`;
- it verifies the result through the normal equations $X^{\mathsf{T}}M^{-1}(Xb - y) = 0$, with M^{-1} applied by an independent LU solve (`getrf`, `getrs`);
- it compares against a literal evaluation with explicit inverses (`getrf`, `getri`, `gemm`, `gemv`).

Table 7 shows the outcome. The speedup (3.7×) is smaller than the FLOP ratio (4.9×): the explicit version consists of large `gemm`-like operations that run at about 140 GFLOP/s (billions of floating-point operations per second), while the Cholesky factorisation and the triangular solves of the derived version run at about 110 GFLOP/s.

10 Formula (1) in a loop: a Kalman measurement update

In applications, Eq. (1) is rarely evaluated once. A sensor array with observation matrix H ($n = 2000$ sensors, $m = 200$ states) and noise covariance R (spd, correlated) delivers a new measurement y_i in every time step, and the estimate

$$b_i = (H^{\mathsf{T}}R^{-1}H)^{-1}H^{\mathsf{T}}R^{-1}y_i$$

is needed in every step. This is the measurement update of a Kalman filter in information form, $b = (P^{-1} + H^{\mathsf{T}}R^{-1}H)^{-1}(P^{-1}b_{\text{prior}} + H^{\mathsf{T}}R^{-1}y)$, with an uninformative prior ($P^{-1} = 0$). `examples/kalman/estimate.mlir` writes the formula inside the loop, as on paper (Fig. 14).

Only y_i changes from step to step, so the pipeline has to do three things, all of which it does without help:

- `loop-invariant-code-motion`, an upstream pass, runs before kernel selection and hoists the `ulm_lam` ops that do not depend on y_i out of the loop. They are `Pure`, so this is safe.
- Kernel selection factorises R and S right after their definitions, i.e. before the loop, and reuses $W = L^{-1}H$, which it computes for $S = W^{\mathsf{T}}W$, for $H^{\mathsf{T}}R^{-1}y_i = W^{\mathsf{T}}(L^{-1}y_i)$ in the loop.
- `ulm-lam-results-to-out-params` turns y_i = observe(i) into observe(i, y), and `buffer-loop-hoisting` moves the buffers of y and b out of the loop.

The loop body that results (Fig. 15) consists of one triangular solve with the 2000×2000 factor L, one `gemv` with W and two triangular solves with the 200×200 factor K: $n^2 + 2nm + 2m^2 \approx 4.9 \cdot 10^6$ FLOPs

per step, against $3.6 \cdot 10^9$ for evaluating the formula from scratch. `examples/kalman/check.sh` checks this structure.

`examples/kalman/driver.cc` compares five versions (Table 8). Every version is timed with 200 steps and with 0 steps (its one-time part), best of 3; the table shows the one-time part, the time per step, and the total for 20 steps.

- *ulm.lam*: the pipeline.
- *ulm.lam without* `loop-invariant-code-motion`: R is still factorised once (it is a function argument), but S is formed and factorised in every step.
- *explicit inverses*: formula (1) evaluated literally in every step, with `getrf` and `getri`. This is the same input ulm.lam gets, so the difference to ulm.lam is mostly the hoisting that ulm.lam does by itself.
- *explicit, hoisted*: written by hand: the invariant inverses R^{-1} and S^{-1} once before the loop, two `gemv` per step.
- *operator G, hoisted*: written by hand: the operator $G = (H^\mathsf{T} R^{-1} H)^{-1} H^\mathsf{T} R^{-1}$ (200×2000) once, one `gemv` per step.

Every estimate is checked through the normal equations with an independent LU solve, and all versions agree.

The operator before the loop, chosen by the trip count. In every step ulm.lam solves with the 2000×2000 factor L ($O(n^2)$ per step); the hand-written version "operator G" forms $G = (H^\mathsf{T} R^{-1} H)^{-1} H^\mathsf{T} R^{-1}$ once ($O(n^2 m)$) and needs one `gemv` ($O(nm)$) per step. Which is better depends on the number of steps. Kernel selection now decides it: for a product in a loop whose factors are loop invariant except the last one, it considers the invariant prefix as an operator P of its own and compares

$$\text{cost}(P) + t \cdot 2\,\text{rows}(P)\,\text{cols}(P)\,\text{cols}(y) \quad \text{with} \quad t \cdot \text{cost}(\text{step})$$

for t steps. $P = S^{-1} H^\mathsf{T} R^{-1}$ itself is not computable in place (it would solve on the transposed view $H^\mathsf{T} L^{-\mathsf{T}}$ of the reused $W = L^{-1} H$), but its transpose is: $P^\mathsf{T} = R^{-1} H S^{-1} = L^{-\mathsf{T}} W S^{-1}$, a `trsm` on a copy of W and two `trsm` with the factor of S; the step is then $P^{\mathsf{T}\mathsf{T}} y$, a `gemv` with `trans`. The flops of a step are matrix–vector flops (`trsv`, `gemv`), which are memory bound and reach about a tenth of the rate of the matrix–matrix flops that form P (on the M5 as on Zen 4); they are weighted with 10. The trip count is a run-time value here, so the generated code computes the condition before the loop, forms P if it holds, and runs one of two loop bodies under the same condition; a constant trip count folds the branch. Both variants compute into the same destination, so the loop still allocates nothing. For $n = 2000$, $m = 200$ the operator pays off from 24 steps on, and at 200 steps ulm.lam is faster than both hand-hoisted versions (Table 8).

10.1 The covariance changes in every step

In many applications the noise covariance itself changes: it is re-estimated online (an adaptive filter), or the noise of the sensors drifts. `examples/kalman/adaptive.mlir` carries R through the loop and updates it in every step by calling an external function (Fig. 16):

$$R_i = \text{update_covariance}(i, R_{i-1}), \qquad b_i = (H^\mathsf{T} R_i^{-1} H)^{-1} H^\mathsf{T} R_i^{-1} y_i.$$

In-place updates by reference. The result of `update_covariance` is marked `{ulm_lam.inout = 1}`: the function updates its argument 1 in place. This extends the by-reference convention of Section 8.1 to external functions that modify a matrix. `ulm-lam-results-to-out-params` turns the declaration into `void update_covariance(size_t, double *)` and every call into `ulm_lamk.call @update_covariance(%i) outs(%R)`. That op is in destination-passing style, like the kernels, so One-Shot Bufferize keeps the loop-carried R in one buffer. A plain call that returns a new matrix would not work here: the loop would have to yield a new allocation in every step, and deallocating it needs ownership bookkeeping at run time. If the matrix is still used after the call, the call updates a copy, so value semantics are preserved. `ulm_lamk.call` reads and writes memory as far as the compiler knows, so it is never hoisted, duplicated or removed.

What runs in every step. Now R_i^{-1} changes in every step, so the Cholesky factorisation of R_i, $W = L_i^{-1}H$, $S = W^\mathsf{T}W$ and its factorisation must run in every step; only what depends on H alone could stay outside. Kernel selection still shares the factorisation of R_i between both occurrences of R_i^{-1} and reuses W for $H^\mathsf{T}R_i^{-1}y_i$ (Fig. 17). Bufferization copies R_i before the in-place factorisation and H before the in-place solve; the caller's R_0 is copied once, before the loop, into the workspace and is not overwritten. There is no allocation inside the loop.

The two copies are necessary with in-place kernels. R_i has two consumers: `potrf` overwrites it, and the next call of `update_covariance` reads all of it, so R_i must exist twice. H is needed in every step, and `trsm` overwrites its right-hand side. A copy followed by the in-place kernel is what an out-of-place variant ($L := \mathrm{chol}(A)$, $W := L^{-1}H$) would do; ulm.lam generates it inline where an input is still needed, so ulmBLAS keeps only the in-place kernels. With the factorisation at about 25 ms per step, the copies are a small part of the time.

`examples/kalman/driver_adaptive.cc` defines an `update_covariance` that moves R towards a base covariance and changes the noise level with the step, which keeps R spd. Over 10 steps, ulm.lam's version takes 338 ms (34 ms per step, dominated by the Cholesky factorisation of the 2000×2000 matrix R_i), against 1177 ms for formula (1) with explicit inverses on the same covariances (3.5×). Its workspace is 67.5 MB: the copy of R that is updated in place, the factor L_i, W, S, y_i and b_i. The worst normal-equation residual over all steps is $9.8 \cdot 10^{-16}$, and both versions agree to $2.1 \cdot 10^{-15}$.

A full Kalman filter also updates the state covariance P and predicts with the state transition. In information form the measurement update is a sum, $P^{-1} \leftarrow P^{-1} + H^\mathsf{T}R^{-1}H$, so it needs inverses as terms of sums and the identity matrix, which kernel selection does not handle yet.

11 Python frontend

`python/ulm/lam/` lets the linear algebra be written in Python and compiled by ulm.lam (Fig. 18): `from ulm import lam`. It has two parts: a frontend that turns Python into `ulm_lam` IR, and a runtime that compiles it and calls the result.

The language. A function decorated with `@lam.function` is traced: it is called with symbolic values, and the Python operators build `ulm_lam` operations (`@` is `ulm_lam.mul`, `+` and `-` are `ulm_lam.add`, a number times a matrix is `ulm_lam.scale`, `.T` is `ulm_lam.transpose`, `.inv` is `ulm_lam.inverse`).

- Parameters have types with properties: `Matrix(n, n, "spd")`, `Vector(n)`, `Index`; a symmetric matrix can be stored as one triangle, `Matrix(n, n, "spd", stored="lower")` (Section 6).
- Sizes are numbers or names: `Matrix("n", "m", "full_rank")` has run-time sizes (Section 8.1). Equal names are equal sizes; they become index parameters in front of the matrices, in order of first appearance. From Python they are taken from the arrays (and checked: `gls: M: size n = 4, but 5 from an earlier argument`); external functions get the sizes they use as parameters too, and Python callbacks receive arrays of the actual shapes. One compiled library serves all sizes.

- The workspace: every library has one (`lib.workspace`, 64-byte aligned), which its functions reuse and which grows as needed; `lib.f(..., work=lam.Workspace(size))` passes another one (`lib.f.workspace_size(...)` is the query). A call from a Python callback gets its own. Intermediate results get their properties from `ulm-lam-infer-properties`.
- Parameters have intents: a matrix is `in` by default; `InOut(Matrix(...))` may be overwritten, and with `lam.update(param, value)` it holds `value` on exit. Return values are `out`.
- `Float` parameters are scalars known only at run time; they scale matrices (`s2 * A`). `lam.eye(n)` is the identity (n a number or a size name), never formed: a term αI of a sum is added to the diagonal. `lam.assume(x, "spd")` asserts properties the compiler cannot derive (e.g. for $K + \sigma^2 I$); a factorisation checks them at run time (`info`). So far a run-time scalar may scale identities only; a scaled matrix or product with one is the next step.
- `with lam.loop(n, carry=[X]) as L:` is a loop with loop-carried matrices (`L.carried`, `L.next(..)`, `L.results`); the formula can be written inside the loop, and `loop-invariant-code-motion` and kernel selection move what does not depend on the loop out of it.
- `lam.extern(name, params, result, updates=k, workspace=True)` declares an external function; a lamp function can also call another one, and their workspaces accumulate.
- Shapes are checked while tracing: a dimension mismatch, or a loop value used outside its loop, is a Python exception at the line that causes it.

The frontend writes the IR as text in the syntax of the dialects; it needs neither the MLIR Python bindings nor any package beyond the Python standard library.

The runtime. `lam.compile(functions)` writes the module, runs `scripts/ulm-lam-compile.sh`, and compiles the C++ with ulmBLAS into a shared library, which it loads with `ctypes`. An unchanged program reuses its library, so recompiling is only needed after a change. Each function of the library is a Python callable whose signature comes from the interface comment the compiler generates (Section 8.1):

- `in` matrices are any C-contiguous `float64` buffer of the right size (NumPy arrays, `array.array`, nested lists); other layouts are copied;
- `inout` matrices must be writable and C-contiguous, because the function may overwrite them;
- `out` matrices are allocated by the runtime and returned (NumPy arrays if NumPy is installed);
- the workspace is queried (`workspace_info = -1`) and allocated by the runtime.

The generated C++ is available from Python: `lib.estimate.cpp` is the code of one function, with its interface comment, and `lib.source` the whole file; the examples print it. External functions can be implemented in Python (`lib.bind(observe=..., record=...)`). The compiler generates a C trampoline for each of them; matrix arguments reach the callback as NumPy views of the caller's memory (or memoryviews), a returned matrix is copied into the caller's buffer, and an exception in a callback is raised again when the compiled function returns.

Results. `python/examples/kalman.py` runs the loop of Fig. 18 with $n = 4000$ sensors, $m = 200$ states and 200 steps, measurements and estimates passing through Python callbacks, and compares it with NumPy (linked to OpenBLAS):

ulm.lam (generated C++, ulmBLAS)	122.0 ms
NumPy, the formula as written in the loop	270271.2 ms
NumPy, the operator G computed by hand before the loop	368.4 ms
relative difference to NumPy	$4.1 \cdot 10^{-15}$

Written as on paper, the Python program is 3× faster than the version a NumPy programmer optimises by hand, and more than 2000 times faster than the formula written literally. Before the operator was formed by kernel selection (with the per-step solves), ulm.lam took 482 ms here and lost against the hand-written version. `python/tests/test_lam.py` (42 tests, `make python`) checks GLS with lists, buffers and NumPy arrays, `inout` parameters, loops with carried matrices and callbacks, the operator chosen by the trip count, functions calling functions, two results, NumPy layouts, one triangle of symmetric matrices, run-time sizes, identity terms with run-time scalars, `info`, exceptions in callbacks, and the errors of the frontend.

The adaptive filter in Python. `python/examples/adaptive_kalman.py` is the adaptive filter of Section 10.1: the covariance R is carried through the loop and updated in place in every step by `update_covariance`, declared with `updates=1` and implemented in Python, where it changes R through a NumPy view of the generated code's memory (Fig. 19).

With $n = 4000$, $m = 200$ and 10 steps the Cholesky factorisation of R_i dominates, and nothing expensive can move out of the loop. NumPy has no Cholesky solve, so the best one can write by hand is one LU solve per step for $[H, y_i]$:

ulm.lam (generated C++, ulmBLAS)	1758.7 ms
NumPy, the formula as written in the loop	8099.1 ms
NumPy, one LU solve per step, by hand	2888.8 ms
relative difference to NumPy	$7.9 \cdot 10^{-15}$

ulm.lam is 1.6× faster than the hand-written NumPy version (Cholesky instead of LU, and $W = L_i^{-1}H$ reused for $H^{\mathsf{T}}R_i^{-1}y_i$) and 4.6× faster than the formula as written. All three times include the Python callback that updates R ($O(n^2)$ per step), and the caller's R_0 (an `in` parameter) is not overwritten.

12 Julia frontend

`julia/UlmLam` is a Julia package with the same frontend as the Python one: functions are traced to `ulm_lam` IR, compiled by the same pipeline, and called with `ccall`; no Python is involved. The notation is Julia's:

```
using UlmLam, LinearAlgebra

@lam function gls(X::Mat(:n, :m, :full_rank), M::Mat(:n, :n, :spd), y::Vec(:n))
    inv(X' * inv(M) * X) * X' * inv(M) * y
end

@lam function gp(K::Mat(:n, :n, :symmetric), s2::Float, Ks::Mat(:n, :m),
                 Kss::Mat(:m, :m, :symmetric), y::Vec(:n))
    Kn = assume(K + s2 * I, :spd)
    (Ks' * inv(Kn) * y, Kss - Ks' * inv(Kn) * Ks)
end

lib = UlmLam.compile([gls, gp])
b = lib.gls(X, M, y)           # Julia arrays in and out
```

`X'`, `inv`, `A \textbackslash {} B`, `B / A`, `I` from `LinearAlgebra`, `assume`, `loop` (a do-block, with loop-carried matrices as its further arguments and results), `extern` with callbacks bound by `bind!`, `InOut`, `Float` and `Index` parameters, sizes as symbols, the workspace, `InfoError` and both backends work as in Python.

Column-major arrays without copies. Julia stores matrices by columns, the generated code by rows. A Julia matrix X ($n \times m$) is, as memory, the row-major matrix X^{T} ($m \times n$). So the frontend declares a

matrix parameter with the transposed type (triangles flipped: a lower triangular X is an upper triangular X^T), the traced function sees `transpose(P)`, and a matrix result is returned transposed; vectors are the same in both orders. Kernel selection turns these transposes into strides, so no matrix is copied for the conversion: GLS from Julia gets exactly the kernel sequence of GLS from Python. That needed four general improvements of the selector, which help Python programs as well: transposes are pushed to the leaves in the normal form ($(AB)^\mathsf{T} = B^\mathsf{T}A^\mathsf{T}$, also for sums and scales), and an `assume` that only drops properties no longer hides a value from the rules (so P and P^T are recognised as transposes of each other for `syrk` and symmetry); a solve from the right with an LU factorisation is `getrs` on the transposed view of the right-hand side (in place); an operand of an in-place solve that is a transposed view is copied transposed (`gecopy`, $O(nm)$, only if nothing cheaper exists); and a transpose used outside the algebra (a loop's initial value, a result) is materialised by `gecopy`. Callbacks of external functions get vectors as `Vector` and matrices as lazy transposes of the row-major buffers, without copies.

Results. `julia/UlmLam/test/runtests.jl` (44 tests) runs GLS with fixed and run-time sizes, solves from the left and from the right (Cholesky and LU), a chain chosen at run time, two results, a loop with callbacks, a loop-carried matrix, GP with `s2 * I` and a wrong assumption (`InfoError`), `InOut`, a given workspace and argument errors, on ulmBLAS and on OpenBLAS (`backend=:cblas`). `julia/examples/gp.jl` and `julia/examples/kalman.jl` are the benchmarks of Section 1: the GP prediction against Julia code written by hand (2–2.4× faster on ulmBLAS, 1.3–1.6× on the same OpenBLAS) and the Kalman loop (3–4× faster than the operator formed by hand).

13 Showcase: Gaussian process regression

A GP (Gaussian process) with kernel k predicts at test points X_* from training data (X, y) by

$$\mu = K_*^\mathsf{T} K^{-1} y, \qquad \Sigma = K_{**} - K_*^\mathsf{T} K^{-1} K_*,$$

with $K = k(X,X) + \sigma^2 I$ ($n \times n$, spd), $K_* = k(X, X_*)$ ($n \times m$) and $K_{**} = k(X_*, X_*)$. Written literally in NumPy, this forms K^{-1}. Libraries therefore implement it by hand; scikit-learn's `GaussianProcessRegressor`, for instance, computes $L = \mathrm{chol}(K)$, $\alpha = L^{-\mathsf{T}}L^{-1}y$, $V = L^{-1}K_*$, $\mu = K_*^\mathsf{T}\alpha$ and $\Sigma = K_{**} - V^\mathsf{T}V$ in `fit` and `predict`. That is the hand work of FLENS-LAPACK again: someone had to notice that $K_*^\mathsf{T}K^{-1}K_* = (L^{-1}K_*)^\mathsf{T}(L^{-1}K_*)$. In ulm.lam the formula is the program (`python/examples/gp.py`):

```
@lam.function
def gp(K: Matrix("n", "n", "symmetric"), s2: Float, Ks: Matrix("n", "m"),
       Kss: Matrix("m", "m", "symmetric"), y: Vector("n")):
    Kn = lam.assume(K + s2 * lam.eye("n"), "spd")
    mean = Ks.T @ Kn.inv @ y
    cov = Kss - Ks.T @ Kn.inv @ Ks
    return mean, cov
```

Here K is the kernel matrix $k(X,X)$ and `s2` the noise variance σ^2, a `Float` parameter known only at run time. `lam.eye("n")` is the identity of the run-time size n; it is never formed: a term αI of a sum becomes the kernel `adddiag` ($A := A + \alpha I$, $O(n)$), with α a constant or a run-time value. Whether $K + \sigma^2 I$ is spd depends on the sign of σ^2, which the compiler cannot know, so the program asserts it with `lam.assume`; the Cholesky factorisation checks the assertion at run time (a negative `s2` gives `info` 1, "Cholesky factorisation of a matrix computed from v3: not positive definite"). Kernel selection derives the hand version: `adddiag` on the copy of K that is factorised, one `potrf`, two `trsv` and a `gemv` for μ, one `trlsm` for $V = L^{-1}K_*$, and `sylrk` with $\beta = 1$ into a copy of K_{**} for Σ, which is then completed (`sycomplete`). For the last step the `syrk` rule of Section 6 was extended: a Gram product $Z^\mathsf{T}Z$ may also be added to a symmetric matrix whose lower triangle is valid ($C = \beta C + \alpha Z^\mathsf{T}Z$). The sizes are run-time names, so one library serves all n and m. Figure 1 in Section 1 shows the generated function. With `backend="cblas"` (Section 8.1) the code is the same, with `ulm_lam_cblas::` instead of `ulmBLAS::` (`python/examples/generated/gp_cblas.cpp`).

Data. Two public data sets, downloaded on first use: the weekly CO_2 concentration at Mauna Loa (NOAA, National Oceanic and Atmospheric Administration, and Scripps; 2718 weeks since 1974), with the kernel and the hyperparameters of Rasmussen and Williams (Gaussian Processes for Machine Learning, 2006, Sec. 5.4.3), and the hourly rentals of a bike sharing system (UCI, University of California, Irvine, Machine Learning Repository; 17 379 hours, 14 features). For CO_2, a back test trained until 2016 predicts the following ten years with an RMSE (root mean square error) of 2.8 ppm, and a forecast from all data reaches 2036 (Fig. 20; the kernel of 2006 underestimates the accelerating trend). The results agree with SciPy and scikit-learn to $2.5 \cdot 10^{-11}$.

Timings. Bike sharing, $m = 1000$ test hours, Apple M5, without computing the kernel matrices (scikit-learn: with them, since it computes them inside). ulm.lam runs twice: on ulmBLAS (with OpenMP, so `potrf` and `trtrs` run as tasks), and with `backend="cblas"` (Section 8.1) on OpenBLAS, the BLAS of NumPy and SciPy here, so that the comparison with the hand-written versions measures the mapping alone:

n	ulm.lam/OpenBLAS	ulm.lam/ulmBLAS	SciPy by hand	NumPy literal	scikit-learn
1000	0.018 s	0.015 s	0.018 s	0.052 s	0.048 s
2000	0.052 s	0.045 s	0.068 s	0.287 s	0.160 s
4000	0.240 s	0.182 s	0.313 s	1.589 s	0.654 s
8000	1.619 s	1.189 s	1.610 s	9.929 s	3.531 s

On the same BLAS, ulm.lam is as fast as SciPy by hand or up to 1.3× faster, 1.5–1.8× faster than scikit-learn and 3–7× faster than the literal formula, and it gets the hand version without anyone writing it. On ulmBLAS it is another 1.3–1.4× faster ($n \geq 4000$): on the M5, ulmBLAS's `potrf` with OpenMP beats OpenBLAS (at $n = 8000$: 0.54 s against 0.67 s).

What the example shows is missing. The kernel matrices $k(X, X)$, $k(X, X_*)$ and $k(X_*, X_*)$ are parameters of `gp`: the Python script `gp.py` computes them before the call, with NumPy (elementwise functions of the data, e.g. $\exp(-\|x_i - x_j\|^2 / 2\ell^2)$). To move them into the ulm.lam program, ulm.lam would need elementwise operations; it could then compute only the triangle that `potrf` reads, obtain squared distances from `syrk`, and write K directly into the buffer that is factorised. Likewise the hyperparameters (ℓ, σ, ...) are fitted outside, by scikit-learn on 1500 points: fitting needs $\log\det K = 2\sum_i \log L_{ii}$ and traces such as $\operatorname{tr}(K^{-1}\partial K)$, scalar-valued operations that would reuse the Cholesky factor L that `gp` already computes. Often only the variances $\operatorname{diag}(\Sigma)$ are needed; after $V = L^{-1}K_*$ they cost $O(nm)$ instead of the $O(nm^2)$ of the whole Σ.

13.1 Large problems: $n = 100\,000$ on bwUniCluster 3.0

`examples/gp_cluster` runs the same function with K as `inout` (factorised in place: at $n = 100\,000$, K has 80 GB) on the 3D Road Network data (UCI; 434 874 points, the altitude predicted from the position, $m = 2000$, Matérn 3/2 kernel), compiled on the cluster for its Zen 4 nodes (2× AMD EPYC 9454, 96 cores; GCC 14.2, `-march=native`, OpenMP). It compares ulm.lam on ulmBLAS, ulm.lam on MKL (`backend=cblas`) and the same steps written by hand with MKL, as SciPy on MKL would do them (times in seconds, 96 threads):

n	RMSE	ulm.lam/ulmBLAS	ulm.lam/MKL	by hand/MKL
4 000	11.03 m	0.44	0.17	0.16
12 500	6.85 m	2.48	1.34	1.46
25 000	4.83 m	13.66	5.71	5.85
50 000	3.17 m	106.5	26.15	27.84
100 000	1.95 m	590.7	153.7	150.7

The mapping is as fast as the hand-written version on the same BLAS (within 6 % either way). In the first measurements, whichever of the two ran first in a process paid about 25 %: the driver zeroed its arrays with the main thread (`std::vector`), so all pages lay on one of the eight NUMA domains, and the kernel's automatic NUMA balancing migrated them during the first heavy run. With the arrays first written by the threads that compute them (the table above), the order no longer matters ($n = 50\,000$, both orders: ulm.lam 26.0 and 26.4 s, by hand 27.4 s; `numactl --interleave=all` has the same effect), and the kernel matrices take 1.1 s instead of 26 s at $n = 100\,000$. *ulmBLAS* is 3.8× slower than MKL at $n = 100\,000$, although it is faster single threaded ($n = 12\,500$: 18.3 s against 22.6 s): its task-parallel `potrf` and `trtrs` scale to about 12 threads (one NUMA domain) and hardly beyond, and `sylrk` runs on one thread. The error of the prediction falls from 15.5 m ($n = 1000$) to 1.95 m: exact GP regression profits from all the data, which makes the $O(n^3)$ factorisation, and a fast `potrf`, the point.

14 Testing strategy

- **Per pass**: `lit/FileCheck` tests on textual IR. Properties are part of the types, so tests of inference rules are one-liners.
- **Property soundness**: random matrices that satisfy the input properties are generated, every inferred property is evaluated numerically, and the test fails on any property that does not hold. This catches wrong inference rules, the most dangerous class of bug.
- **Numerical** (implemented): every example is compiled to C++ and compared with an independent evaluation. `examples/cases/` covers the LU solve, plain and transposed, upper triangular solves, right solves with L^{-1} and $L^{-\mathsf{T}}$, a Cholesky solve with a vector, a sum of three plain terms and $A^{-1}BC + D$, against naive triple loops and Gaussian elimination; all agree to $\leq 6 \cdot 10^{-16}$. It also checks that inputs passed by value are unchanged.
- **Structural** (implemented, scripts): kernel selection and memory placement are checked on the generated IR, e.g. two Cholesky factorisations and three copies for GLS, no allocation in the loop.
- **Differential**: `convert-ulm-lamk-to-linalg` against `convert-ulm-lamk-to-ulmblas`.
- **Quality**: the LAMP benchmark [1] (`https://github.com/HPAC/LAMP_benchmark`) and Linnea's 25 application problems. Each derivation is compared against Linnea's FLOP count and run time.

15 Status and plan

Done.

- ODS for both dialects, interfaces and passes, checked by all `mlir-tblgen` backends, and a kernel catalogue whose TableGen syntax is checked;
- the `ulm_lam` and `ulm_lamk` dialects, generated from the ODS and loaded into `mlir-opt` as a plugin: property inference, relaxed type compatibility, verifiers, canonicalisation patterns, kernel costs and bufferization;
- `ulm-lam-normalize`, `ulm-lam-infer-properties`, `ulm-lam-select-kernels` with `strategy=gmc` including inverses, factorisations, solves, $Z^{\mathsf{T}}Z$ and sums (Section 7.4), and `ulm-lam-strip-properties`;
- parameter intents `in`, `inout`, `out`, and a generated comment that documents them for the caller;
- `ulm-lam-results-to-out-params`: matrices by reference, the caller owns them ($X = \text{foo}(i)$ becomes $\text{foo}(i, X)$, in/out results are updated in place, also by external functions via `ulm_lamk.call`, inputs passed by value are read-only);

- `ulm-lam-workspace`: temporaries from a caller-provided workspace (query, allocate, or check), with queries accumulated over calls of generated and external functions;
- `convert-ulm-lamk-to-ulmblas` for all kernels kernel selection emits;
- the Python frontend and runtime (`from ulm import lam`): tracing, loops, external functions as Python callbacks, workspaces, NumPy or plain buffers;
- GLS from its normal form, formula (1) in a loop (with a constant and with an updated covariance), the loop example and eight kernel selection cases, compiled to C++ with ulmBLAS by passes alone and verified numerically; the hand-written GLS stages 1–4 parse and verify.

Compared with Linnea. Linnea itself has not been run for this comparison, and the comparison on Linnea's 25 application problems is still to be done (see below), so it compares features, and the one case where both have been compared.

- *Linnea is ahead in the mapping itself.* It searches a graph of alternative derivations (best-first, anytime), with choices of factorisations and common subexpressions across statements; ulm.lam has `strategy=gmc` only: per product the generalised matrix chain (Linnea's inner algorithm), plus reuse of computed intervals, `syrk` for $Z^\mathsf{T}Z$, inverses of products factor by factor, and the operator before a loop. Linnea knows more factorisations and kernels (QR, eigenvalue decomposition, diagonal matrices, `trmm`) and more rewrite rules; ulm.lam has Cholesky and LU, `symm`, `symv`, `syrk`, triangular solves and `adddiag`. Linnea was evaluated on 25 applications and 100 random expressions against Julia, Armadillo, Eigen and Matlab; ulm.lam on GLS, the Kalman update, GP regression and the cases of `examples/cases`.
- *Equal where compared:* for GLS ulm.lam derives exactly Linnea's kernel sequence (Table 6).
- *ulm.lam goes beyond what Linnea generates* (straight-line Julia code for fixed sizes): sizes known only at run time, with the plan chosen at run time; loops, with invariant work moved out and the operator formed before the loop by the trip count; memory planned by bufferization (in-place kernels, `inout`, a LAPACK-style workspace with query, no allocation in loops); `info` instead of aborting; symmetric matrices stored as one triangle; scalars known at run time; a Python frontend with callbacks; two backends (ulmBLAS, CBLAS/LAPACK); problems up to $n = 100\,000$ (Section 13.1).

In short: in the breadth of the mapping (search, factorisations, kernel catalogue) Linnea is clearly ahead; in what a compiler needs to be used (sizes, loops, memory, errors, integration into a language) ulm.lam is.

Plan.

M1 *Largely done.* Dialects, verifiers, property closure and inference. Missing: a CMake build with a `ulm-lam-opt` tool and `lit` tests in place of the scripts.

M2 *Partly done.* `ulm-lam-normalize` lacks pushing transposes and inverses to the leaves and hoisting scales.

M3 *Done for what kernel selection emits.* `ulm_lamk` with DPS, bufferization and the ulmBLAS lowering. Missing: the lowering of `trmm`.

M4 *Partly done.* `strategy=gmc` with gemm, gemv, solves, factorisations and $Z^\mathsf{T}Z$. `syrk` and `symm`/`symv` with triangles tracked (Section 6). Missing: `gemmtr` for symmetric products that are not $Z^\mathsf{T}Z$, triangular matrices stored as one triangle (`trmm`), the catalogue backend, so that new kernels are added as records, and a fallback plan where a factorisation fails (Section 8.1).

M5 `strategy=search`: sharing across products beyond reuse of computed intervals, factorisation choices, anytime search; the LAMP benchmark; a full Kalman filter (Section 10).

M6 *Done:* loop-aware selection: a loop-invariant prefix of a product is formed before the loop if the trip count, known at run time, makes it pay off (the operator of Section 10). Open: invariant subchains other than a prefix, and a measured instead of a fixed weight for matrix–vector flops.

M7 *Done for Python* (Section 11): a frontend that traces Python to `ulm_lam` IR, with a runtime and callbacks. Open: `strategy=egraph` experiments; QR and LDL^T once ulmBLAS has them (Section 8.1).

Next steps, in this order.

1. *A Julia binding*: a frontend that traces Julia functions to `ulm_lam` IR, as the Python one does, and calls the generated library with `ccall`. Julia is where a linear algebra compiler finds its community, and ulmBLAS aims at being a BLAS for Julia as well.
2. *More showcases* that ulm.lam can do now or nearly: ridge regression $(X^\mathsf{T}X + \lambda I)^{-1}X^\mathsf{T}y$ (`syrk`, `adddiag`, Cholesky); linear attention, where the order of $QK^\mathsf{T}V$ is chosen at run time from the sizes; Bayesian linear regression and the full Kalman filter (predict and update), which need run-time scalars on products and `gemmtr`.
3. *Linnea's 25 application problems and the LAMP benchmark* (`https://github.com/HPAC/LAMP_benchmark`): run them through ulm.lam and compare FLOPs and coverage per problem with Linnea. That gives the precise list of missing rules, factorisations and kernels, and the basis for `strategy=search`.

Open questions.

- Dynamic sizes: should the compiler use size hints or multi-version code with run-time dispatch?
- Cost model: FLOPs mispredict memory-bound kernels. Measured per-kernel efficiency (`cost-model=measured`) is the obvious next step.
- Should storage formats (packed triangular, banded) become a second encoding dimension, or a separate type?
- Batched and tensor contractions: should `ulm_lam` be generalised, or interoperate with `linalg`?

A Generated code

`make` compiles every example with `scripts/ulm-lam-compile.sh` and keeps the C++ in `examples/*/generated/`. The files are reproduced here in full, as generated (EmitC's variable names, formatted with `clang-format`).

A.1 `examples/gls/generated/gls.cpp`

Generalised least squares from its normal form (Section 9).

```
#include <cstddef>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <ulmblas/lapack/potrf.h>
#include <ulmblas/lapack/trtrs.h>
#include <ulmblas/ulmblas.h>
static void
ulm_lam_copy(void *dst, const void *src, std::size_t bytes)
{
```

```
#ifdef _OPENMP
    const std::size_t chunk = std::size_t(1) << 20;
    if (bytes >= 8 * chunk) {
        std::ptrdiff_t chunks = (bytes + chunk - 1) / chunk;
#pragma omp parallel for schedule(static)
        for (std::ptrdiff_t c = 0; c < chunks; ++c) {
            std::size_t off = c * chunk;
            std::size_t len = bytes - off < chunk ? bytes - off : chunk;
            std::memcpy(static_cast<char *>(dst) + off,
                        static_cast<const char *>(src) + off, len);
        }
        return;
    }
#endif
    std::memcpy(dst, src, bytes);
}

/* gls: matrices by reference (row-major), owned by the caller
 *   v1  double[2000][200]   in    read only
 *   v2  double[2000][2000]  in    read only
 *   v3  double[2000][1]     in    read only
 *   v4  double[200][1]      out   written
 *   v5  int8_t *            work  workspace (bytes)
 *   v6  int64_t *           info  -1: query, 0: allocate, >0: size of work
 * returns info:
 *     0  success
 *     1  Cholesky factorisation of v2: not positive definite
 *     2  Cholesky factorisation of a computed matrix: not positive definite
 *    -6  v6: workspace too small
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
gls(double *v1, double *v2, double *v3, double *v4, int8_t *v5, int64_t *v6)
{
    int64_t v7 = 35536000;
    double v8 = 1.00000000000000000e+00;
    double v9 = 0.0e+00;
    int64_t v10 = 0;
    size_t v11 = 0;
    int64_t v12 = v6[v11];
    int64_t v13;
    if (v12 == -1) {
        v6[v11] = v7;
        v13 = v10;
    } else {
        int64_t v14;
        if (v12 > v10 & v12 < v7) {
            v14 = -6;
        } else {
            bool v15 = v12 == v10;
            int8_t *v16;
            if (v15) {
                void *v17 = aligned_alloc(64, 35536000);
                v16 = (int8_t *)v17;
            } else {
                v16 = v5;
            }
            int8_t *v18 = v16;
            int64_t v19 = (int64_t)v11;
            double *v20 = (double *)(v18 + v19);
            ulm_lam_copy(v20, v2, 32000000);
            int64_t v21 =
                ulmBLAS::potrf<std::ptrdiff_t>(true, 2000, v20, 2000, 1);
            bool v22 = (bool)(v21 != v10);
            uint64_t v23 = (uint64_t)v22;
            int64_t v24 = (int64_t)v23;
            int64_t v25;
            if (v24 == v10) {
                int64_t v26 = (int64_t)32000000;
                double *v27 = (double *)(v18 + v26);
                int64_t v28 = (int64_t)32320000;
                double *v29 = (double *)(v18 + v28);
                ulm_lam_copy(v29, v1, 3200000);
```

```
                (void)ulmBLAS::trtrs<std::ptrdiff_t>(true, false, false, false,
                                                     2000, 200, v20, 2000, 1,
                                                     v29, 200, 1);
                ulmBLAS::sylrk<std::ptrdiff_t>(200, 2000, v8, v29, 1, 200, v9,
                                               v27, 200, 1);
                int64_t v30 =
                    ulmBLAS::potrf<std::ptrdiff_t>(true, 200, v27, 200, 1);
                int64_t v31 = v30 != v10 ? 2 : v10;
                if (v31 == v10) {
                    int64_t v32 = (int64_t)35520000;
                    double *v33 = (double *)(v18 + v32);
                    ulm_lam_copy(v33, v3, 16000);
                    ulmBLAS::trlsv<std::ptrdiff_t>(2000, false, v20, 2000, 1,
                                                   v33, 1);
                    ulmBLAS::gemv<std::ptrdiff_t>(200, 2000, v8, v29, 1, 200,
                                                  v33, 1, v9, v4, 1);
                    ulmBLAS::trlsv<std::ptrdiff_t>(200, false, v27, 200, 1, v4,
                                                   1);
                    ulmBLAS::trusv<std::ptrdiff_t>(200, false, v27, 1, 200, v4,
                                                   1);
                }
                v25 = v31;
            } else {
                v25 = v24;
            }
            int64_t v34 = v25;
            if (v15) {
                free(v18);
            }
            v14 = v34;
        }
        v13 = v14;
    }
    return v13;
}
```

A.2 `examples/gls/generated/gls_inout.cpp`

GLS with M and y declared `inout` (Section 8.1).

```
#include <cstddef>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <ulmblas/lapack/potrf.h>
#include <ulmblas/lapack/trtrs.h>
#include <ulmblas/ulmblas.h>
static void
ulm_lam_copy(void *dst, const void *src, std::size_t bytes)
{
#ifdef _OPENMP
    const std::size_t chunk = std::size_t(1) << 20;
    if (bytes >= 8 * chunk) {
        std::ptrdiff_t chunks = (bytes + chunk - 1) / chunk;
#pragma omp parallel for schedule(static)
        for (std::ptrdiff_t c = 0; c < chunks; ++c) {
            std::size_t off = c * chunk;
            std::size_t len = bytes - off < chunk ? bytes - off : chunk;
            std::memcpy(static_cast<char *>(dst) + off,
                        static_cast<const char *>(src) + off, len);
        }
        return;
    }
#endif
    std::memcpy(dst, src, bytes);
}

/* gls_inout: matrices by reference (row-major), owned by the caller
 *   v1  double[2000][200]   in     read only
 *   v2  double[2000][2000]  inout  may be overwritten (unspecified on exit)
 *   v3  double[2000][1]     inout  may be overwritten (unspecified on exit)
```

```
 *   v4  double[200][1]      out    written
 *   v5  int8_t *            work   workspace (bytes)
 *   v6  int64_t *           info   -1: query, 0: allocate, >0: size of work
 * returns info:
 *     0  success
 *     1  Cholesky factorisation of v2: not positive definite
 *     2  Cholesky factorisation of a computed matrix: not positive definite
 *    -6  v6: workspace too small
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
gls_inout(double *v1, double *v2, double *v3, double *v4, int8_t *v5,
          int64_t *v6)
{
    int64_t v7 = 3520000;
    double v8 = 1.00000000000000000e+00;
    double v9 = 0.0e+00;
    int64_t v10 = 0;
    size_t v11 = 0;
    int64_t v12 = v6[v11];
    int64_t v13;
    if (v12 == -1) {
        v6[v11] = v7;
        v13 = v10;
    } else {
        int64_t v14;
        if (v12 > v10 & v12 < v7) {
            v14 = -6;
        } else {
            bool v15 = v12 == v10;
            int8_t *v16;
            if (v15) {
                void *v17 = aligned_alloc(64, 3520000);
                v16 = (int8_t *)v17;
            } else {
                v16 = v5;
            }
            int8_t *v18 = v16;
            int64_t v19 =
                ulmBLAS::potrf<std::ptrdiff_t>(true, 2000, v2, 2000, 1);
            bool v20 = (bool)(v19 != v10);
            uint64_t v21 = (uint64_t)v20;
            int64_t v22 = (int64_t)v21;
            int64_t v23;
            if (v22 == v10) {
                int64_t v24 = (int64_t)v11;
                double *v25 = (double *)(v18 + v24);
                int64_t v26 = (int64_t)320000;
                double *v27 = (double *)(v18 + v26);
                ulm_lam_copy(v27, v1, 3200000);
                (void)ulmBLAS::trtrs<std::ptrdiff_t>(true, false, false, false,
                                                     2000, 200, v2, 2000, 1,
                                                     v27, 200, 1);
                ulmBLAS::sylrk<std::ptrdiff_t>(200, 2000, v8, v27, 1, 200, v9,
                                               v25, 200, 1);
                int64_t v28 =
                    ulmBLAS::potrf<std::ptrdiff_t>(true, 200, v25, 200, 1);
                int64_t v29 = v28 != v10 ? 2 : v10;
                if (v29 == v10) {
                    ulmBLAS::trlsv<std::ptrdiff_t>(2000, false, v2, 2000, 1, v3,
                                                   1);
                    ulmBLAS::gemv<std::ptrdiff_t>(200, 2000, v8, v27, 1, 200,
                                                  v3, 1, v9, v4, 1);
                    ulmBLAS::trlsv<std::ptrdiff_t>(200, false, v25, 200, 1, v4,
                                                   1);
                    ulmBLAS::trusv<std::ptrdiff_t>(200, false, v25, 1, 200, v4,
                                                   1);
                }
                v23 = v29;
            } else {
                v23 = v22;
            }
            int64_t v30 = v23;
```

```
            if (v15) {
                free(v18);
            }
            v14 = v30;
        }
        v13 = v14;
    }
    return v13;
}
```

A.3 `examples/loop/generated/accumulate.cpp`

X += ABC in a loop (Section 8.1).

```
#include <cstddef>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <ulmblas/ulmblas.h>
extern "C" void
foo(size_t, double *);
/* accumulate: matrices by reference (row-major), owned by the caller
 *   v1  double[500][500]  in     read only
 *   v2  double[500][500]  in     read only
 *   v3  double[500][500]  inout  updated in place
 *   v4  size_t            in
 *   v5  int8_t *          work   workspace (bytes)
 *   v6  int64_t *         info   -1: query, 0: allocate, >0: size of work
 * returns info:
 *     0  success
 *    -6  v6: workspace too small
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
accumulate(double *v1, double *v2, double *v3, size_t v4, int8_t *v5,
           int64_t *v6)
{
    int64_t v7 = 4000000;
    double v8 = 1.00000000000000000e+00;
    double v9 = 0.0e+00;
    int64_t v10 = 0;
    size_t v11 = 0;
    int64_t v12 = v6[v11];
    int64_t v13;
    if (v12 == -1) {
        v6[v11] = v7;
        v13 = v10;
    } else {
        bool v14 = v12 > v10 & v12 < v7;
        if (v14) {
        } else {
            bool v15 = v12 == v10;
            int8_t *v16;
            if (v15) {
                void *v17 = aligned_alloc(64, 4000000);
                v16 = (int8_t *)v17;
            } else {
                v16 = v5;
            }
            int8_t *v18 = v16;
            int64_t v19 = (int64_t)v11;
            double *v20 = (double *)(v18 + v19);
            int64_t v21 = (int64_t)2000000;
            double *v22 = (double *)(v18 + v21);
            for (size_t i23 = v11; i23 < v4; i23 += 1) {
                foo(i23, v20);
                ulmBLAS::gemm<std::ptrdiff_t>(500, 500, 500, v8, false, v1, 500,
                                              1, false, v20, 500, 1, v9, v22,
                                              500, 1);
                ulmBLAS::gemm<std::ptrdiff_t>(500, 500, 500, v8, false, v22,
                                              500, 1, false, v2, 500, 1, v8, v3,
```

```
                                            500, 1);
            }
            if (v15) {
                free(v18);
            }
        }
        v13 = v14 ? -6 : v10;
    }
    return v13;
}
```

A.4 `examples/kalman/generated/estimate.cpp`

Formula (1) in a loop with a constant covariance (Section 10).

```
#include <cstddef>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <ulmblas/lapack/potrf.h>
#include <ulmblas/lapack/trtrs.h>
#include <ulmblas/ulmblas.h>
static void
ulm_lam_copy(void *dst, const void *src, std::size_t bytes)
{
#ifdef _OPENMP
    const std::size_t chunk = std::size_t(1) << 20;
    if (bytes >= 8 * chunk) {
        std::ptrdiff_t chunks = (bytes + chunk - 1) / chunk;
#pragma omp parallel for schedule(static)
        for (std::ptrdiff_t c = 0; c < chunks; ++c) {
            std::size_t off = c * chunk;
            std::size_t len = bytes - off < chunk ? bytes - off : chunk;
            std::memcpy(static_cast<char *>(dst) + off,
                        static_cast<const char *>(src) + off, len);
        }
        return;
    }
#endif
    std::memcpy(dst, src, bytes);
}

template <typename T>
static void
ulm_lam_gecopy(std::ptrdiff_t m, std::ptrdiff_t n, const T *A,
               std::ptrdiff_t incRowA, std::ptrdiff_t incColA, T *B,
               std::ptrdiff_t incRowB, std::ptrdiff_t incColB)
{
    const std::ptrdiff_t bs = 64;
#pragma omp parallel for collapse(2) schedule(static) if (m * n > 1 << 16)
    for (std::ptrdiff_t i0 = 0; i0 < m; i0 += bs)
        for (std::ptrdiff_t j0 = 0; j0 < n; j0 += bs)
            for (std::ptrdiff_t i = i0; i < i0 + bs && i < m; ++i)
                for (std::ptrdiff_t j = j0; j < j0 + bs && j < n; ++j)
                    B[i * incRowB + j * incColB] = A[i * incRowA + j * incColA];
}

extern "C" void
observe(size_t, double *);
extern "C" void
record(size_t, double *);
/* estimate: matrices by reference (row-major), owned by the caller
 *   v1  double[2000][200]   in    read only
 *   v2  double[2000][2000]  in    read only
 *   v3  size_t              in
 *   v4  int8_t *            work  workspace (bytes)
 *   v5  int64_t *           info  -1: query, 0: allocate, >0: size of work
 * returns info:
 *     0  success
 *     1  Cholesky factorisation of v2: not positive definite
 *     2  Cholesky factorisation of a computed matrix: not positive definite
```

```c
 *    -5  v5: workspace too small
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
estimate(double *v1, double *v2, size_t v3, int8_t *v4, int64_t *v5)
{
    int64_t v6 = 38737600;
    double v7 = 1.00000000000000000e+00;
    double v8 = 0.0e+00;
    int64_t v9 = 0;
    size_t v10 = 0;
    int64_t v11 = v5[v10];
    int64_t v12;
    if (v11 == -1) {
        v5[v10] = v6;
        v12 = v9;
    } else {
        int64_t v13;
        if (v11 > v9 & v11 < v6) {
            v13 = -5;
        } else {
            bool v14 = v11 == v9;
            int8_t *v15;
            if (v14) {
                void *v16 = aligned_alloc(64, 38737600);
                v15 = (int8_t *)v16;
            } else {
                v15 = v4;
            }
            int8_t *v17 = v15;
            int64_t v18 = (int64_t)v10;
            double *v19 = (double *)(v17 + v18);
            ulm_lam_copy(v19, v2, 32000000);
            int64_t v20 =
                ulmBLAS::potrf<std::ptrdiff_t>(true, 2000, v19, 2000, 1);
            bool v21 = (bool)(v20 != v9);
            uint64_t v22 = (uint64_t)v21;
            int64_t v23 = (int64_t)v22;
            int64_t v24;
            if (v23 == v9) {
                int64_t v25 = (int64_t)32000000;
                double *v26 = (double *)(v17 + v25);
                int64_t v27 = (int64_t)32320000;
                double *v28 = (double *)(v17 + v27);
                ulm_lam_copy(v28, v1, 3200000);
                (void)ulmBLAS::trtrs<std::ptrdiff_t>(true, false, false, false,
                                                     2000, 200, v19, 2000, 1,
                                                     v28, 200, 1);
                ulmBLAS::sylrk<std::ptrdiff_t>(200, 2000, v7, v28, 1, 200, v8,
                                               v26, 200, 1);
                int64_t v29 =
                    ulmBLAS::potrf<std::ptrdiff_t>(true, 200, v26, 200, 1);
                int64_t v30 = v29 != v9 ? 2 : v9;
                if (v30 == v9) {
                    ptrdiff_t v31 = (ptrdiff_t)v3;
                    ptrdiff_t v32 = (ptrdiff_t)v10;
                    bool v33 = (v31 > v32 ? v3 : v10) * 8000000 + 960400000 <
                               (v31 > v32 ? v3 : v10) * 48800000;
                    int64_t v34 = (int64_t)35520000;
                    double *v35 = (double *)(v17 + v34);
                    if (v33) {
                        ulm_lam_gecopy(200, 2000, v28, 1, 200, v35, 2000, 1);
                        (void)ulmBLAS::trtrs<std::ptrdiff_t>(
                            true, false, false, false, 200, 2000, v26, 200, 1,
                            v35, 2000, 1);
                        (void)ulmBLAS::trtrs<std::ptrdiff_t>(
                            false, false, false, false, 200, 2000, v26, 1, 200,
                            v35, 2000, 1);
                        (void)ulmBLAS::trtrs<std::ptrdiff_t>(
                            false, false, false, false, 2000, 200, v19, 1, 2000,
                            v35, 1, 2000);
                    }
                    int64_t v36 = (int64_t)38720000;
```

```
                    double *v37 = (double *)(v17 + v36);
                    int64_t v38 = (int64_t)38736000;
                    double *v39 = (double *)(v17 + v38);
                    for (size_t i40 = v10; i40 < v3; i40 += 1) {
                        observe(i40, v37);
                        if (v33) {
                            ulmBLAS::gemv<std::ptrdiff_t>(200, 2000, v7, v35,
                                                          2000, 1, v37, 1, v8,
                                                          v39, 1);
                        } else {
                            ulmBLAS::trlsv<std::ptrdiff_t>(2000, false, v19,
                                                           2000, 1, v37, 1);
                            ulmBLAS::gemv<std::ptrdiff_t>(
                                200, 2000, v7, v28, 1, 200, v37, 1, v8, v39, 1);
                            ulmBLAS::trlsv<std::ptrdiff_t>(200, false, v26, 200,
                                                           1, v39, 1);
                            ulmBLAS::trusv<std::ptrdiff_t>(200, false, v26, 1,
                                                           200, v39, 1);
                        }
                        record(i40, v39);
                    }
                }
                v24 = v30;
            } else {
                v24 = v23;
            }
            int64_t v41 = v24;
            if (v14) {
                free(v17);
            }
            v13 = v41;
        }
        v12 = v13;
    }
    return v12;
}
```

A.5 `examples/kalman/generated/adaptive.cpp`

Formula (1) in a loop with the covariance updated in every step (Section 10.1).

```
#include <cstddef>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <ulmblas/lapack/potrf.h>
#include <ulmblas/lapack/trtrs.h>
#include <ulmblas/ulmblas.h>
static void
ulm_lam_copy(void *dst, const void *src, std::size_t bytes)
{
#ifdef _OPENMP
    const std::size_t chunk = std::size_t(1) << 20;
    if (bytes >= 8 * chunk) {
        std::ptrdiff_t chunks = (bytes + chunk - 1) / chunk;
#pragma omp parallel for schedule(static)
        for (std::ptrdiff_t c = 0; c < chunks; ++c) {
            std::size_t off = c * chunk;
            std::size_t len = bytes - off < chunk ? bytes - off : chunk;
            std::memcpy(static_cast<char *>(dst) + off,
                        static_cast<const char *>(src) + off, len);
        }
        return;
    }
#endif
    std::memcpy(dst, src, bytes);
}

extern "C" void
update_covariance(size_t, double *);
extern "C" void
```

```
observe(size_t, double *);
extern "C" void
record(size_t, double *);
/* estimate_adaptive: matrices by reference (row-major), owned by the caller
 *   v1  double[2000][200]   in    read only
 *   v2  double[2000][2000]  in    read only
 *   v3  size_t              in
 *   v4  int8_t *            work  workspace (bytes)
 *   v5  int64_t *           info  -1: query, 0: allocate, >0: size of work
 * returns info:
 *     0  success
 *     1  Cholesky factorisation of a computed matrix: not positive definite
 *     2  Cholesky factorisation of a computed matrix: not positive definite
 *    -5  v5: workspace too small
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
estimate_adaptive(double *v1, double *v2, size_t v3, int8_t *v4, int64_t *v5)
{
    int64_t v6 = 67537600;
    double v7 = 1.0000000000000000e+00;
    double v8 = 0.0e+00;
    int64_t v9 = 0;
    size_t v10 = 0;
    int64_t v11 = v5[v10];
    int64_t v12;
    if (v11 == -1) {
        v5[v10] = v6;
        v12 = v9;
    } else {
        int64_t v13;
        if (v11 > v9 & v11 < v6) {
            v13 = -5;
        } else {
            bool v14 = v11 == v9;
            int8_t *v15;
            if (v14) {
                void *v16 = aligned_alloc(64, 67537600);
                v15 = (int8_t *)v16;
            } else {
                v15 = v4;
            }
            int8_t *v17 = v15;
            int64_t v18 = (int64_t)v10;
            double *v19 = (double *)(v17 + v18);
            ulm_lam_copy(v19, v2, 32000000);
            int64_t v20 = (int64_t)32000000;
            double *v21 = (double *)(v17 + v20);
            int64_t v22 = (int64_t)64000000;
            double *v23 = (double *)(v17 + v22);
            int64_t v24 = (int64_t)64016000;
            double *v25 = (double *)(v17 + v24);
            int64_t v26 = (int64_t)64336000;
            double *v27 = (double *)(v17 + v26);
            int64_t v28 = (int64_t)67536000;
            double *v29 = (double *)(v17 + v28);
            int64_t v30;
            v30 = v9;
            for (size_t i31 = v10; i31 < v3; i31 += 1) {
                int64_t v32 = v30;
                int64_t v33;
                if (v32 == v9) {
                    update_covariance(i31, v19);
                    ulm_lam_copy(v21, v19, 32000000);
                    int64_t v34 = ulmBLAS::potrf<std::ptrdiff_t>(true, 2000,
                                                                 v21, 2000, 1);
                    bool v35 = (bool)(v34 != v9);
                    uint64_t v36 = (uint64_t)v35;
                    int64_t v37 = (int64_t)v36;
                    int64_t v38;
                    if (v37 == v9) {
                        observe(i31, v23);
                        ulm_lam_copy(v27, v1, 3200000);
```

```
                        (void)ulmBLAS::trtrs<std::ptrdiff_t>(
                            true, false, false, false, 2000, 200, v21, 2000, 1,
                            v27, 200, 1);
                        ulmBLAS::sylrk<std::ptrdiff_t>(200, 2000, v7, v27, 1,
                                                       200, v8, v25, 200, 1);
                        int64_t v39 = ulmBLAS::potrf<std::ptrdiff_t>(
                            true, 200, v25, 200, 1);
                        int64_t v40 = v39 != v9 ? 2 : v9;
                        if (v40 == v9) {
                            ulmBLAS::trlsv<std::ptrdiff_t>(2000, false, v21,
                                                           2000, 1, v23, 1);
                            ulmBLAS::gemv<std::ptrdiff_t>(
                                200, 2000, v7, v27, 1, 200, v23, 1, v8, v29, 1);
                            ulmBLAS::trlsv<std::ptrdiff_t>(200, false, v25, 200,
                                                           1, v29, 1);
                            ulmBLAS::trusv<std::ptrdiff_t>(200, false, v25, 1,
                                                           200, v29, 1);
                            record(i31, v29);
                        }
                        v38 = v40;
                    } else {
                        v38 = v37;
                    }
                    v33 = v38;
                } else {
                    v33 = v32;
                }
                v30 = v33;
            }
            int64_t v41 = v30;
            if (v14) {
                free(v17);
            }
            v13 = v41;
        }
        v12 = v13;
    }
    return v12;
}
```

A.6 `examples/compose/generated/compose.cpp`

Functions with workspaces calling functions with workspaces (Section 8.1).

```
#include <cstddef>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <ulmblas/lapack/potrf.h>
#include <ulmblas/lapack/trtrs.h>
#include <ulmblas/ulmblas.h>
static void
ulm_lam_copy(void *dst, const void *src, std::size_t bytes)
{
#ifdef _OPENMP
    const std::size_t chunk = std::size_t(1) << 20;
    if (bytes >= 8 * chunk) {
        std::ptrdiff_t chunks = (bytes + chunk - 1) / chunk;
#pragma omp parallel for schedule(static)
        for (std::ptrdiff_t c = 0; c < chunks; ++c) {
            std::size_t off = c * chunk;
            std::size_t len = bytes - off < chunk ? bytes - off : chunk;
            std::memcpy(static_cast<char *>(dst) + off,
                        static_cast<const char *>(src) + off, len);
        }
        return;
    }
#endif
    std::memcpy(dst, src, bytes);
}
```

```
extern "C" void
observe(size_t, double *);
extern "C" void
postprocess(size_t, double *, int8_t *, int64_t *);
/* gls: matrices by reference (row-major), owned by the caller
 *   v1  double[2000][200]   in    read only
 *   v2  double[2000][2000]  in    read only
 *   v3  double[2000][1]     in    read only
 *   v4  double[200][1]      out   written
 *   v5  int8_t *            work  workspace (bytes)
 *   v6  int64_t *           info  -1: query, 0: allocate, >0: size of work
 * returns info:
 *     0  success
 *     1  Cholesky factorisation of v2: not positive definite
 *     2  Cholesky factorisation of a computed matrix: not positive definite
 *    -6  v6: workspace too small
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
gls(double *v1, double *v2, double *v3, double *v4, int8_t *v5, int64_t *v6)
{
    int64_t v7 = 35536000;
    double v8 = 1.00000000000000000e+00;
    double v9 = 0.0e+00;
    int64_t v10 = 0;
    size_t v11 = 0;
    int64_t v12 = v6[v11];
    int64_t v13;
    if (v12 == -1) {
        v6[v11] = v7;
        v13 = v10;
    } else {
        int64_t v14;
        if (v12 > v10 & v12 < v7) {
            v14 = -6;
        } else {
            bool v15 = v12 == v10;
            int8_t *v16;
            if (v15) {
                void *v17 = aligned_alloc(64, 35536000);
                v16 = (int8_t *)v17;
            } else {
                v16 = v5;
            }
            int8_t *v18 = v16;
            int64_t v19 = (int64_t)v11;
            double *v20 = (double *)(v18 + v19);
            ulm_lam_copy(v20, v2, 32000000);
            int64_t v21 =
                ulmBLAS::potrf<std::ptrdiff_t>(true, 2000, v20, 2000, 1);
            bool v22 = (bool)(v21 != v10);
            uint64_t v23 = (uint64_t)v22;
            int64_t v24 = (int64_t)v23;
            int64_t v25;
            if (v24 == v10) {
                int64_t v26 = (int64_t)32000000;
                double *v27 = (double *)(v18 + v26);
                int64_t v28 = (int64_t)32320000;
                double *v29 = (double *)(v18 + v28);
                ulm_lam_copy(v29, v1, 3200000);
                (void)ulmBLAS::trtrs<std::ptrdiff_t>(true, false, false, false,
                                                     2000, 200, v20, 2000, 1,
                                                     v29, 200, 1);
                ulmBLAS::sylrk<std::ptrdiff_t>(200, 2000, v8, v29, 1, 200, v9,
                                               v27, 200, 1);
                int64_t v30 =
                    ulmBLAS::potrf<std::ptrdiff_t>(true, 200, v27, 200, 1);
                int64_t v31 = v30 != v10 ? 2 : v10;
                if (v31 == v10) {
                    int64_t v32 = (int64_t)35520000;
                    double *v33 = (double *)(v18 + v32);
                    ulm_lam_copy(v33, v3, 16000);
                    ulmBLAS::trlsv<std::ptrdiff_t>(2000, false, v20, 2000, 1,
```

```
                                                v33, 1);
                    ulmBLAS::gemv<std::ptrdiff_t>(200, 2000, v8, v29, 1, 200,
                                                  v33, 1, v9, v4, 1);
                    ulmBLAS::trlsv<std::ptrdiff_t>(200, false, v27, 200, 1, v4,
                                                   1);
                    ulmBLAS::trusv<std::ptrdiff_t>(200, false, v27, 1, 200, v4,
                                                   1);
                }
                v25 = v31;
            } else {
                v25 = v24;
            }
            int64_t v34 = v25;
            if (v15) {
                free(v18);
            }
            v14 = v34;
        }
        v13 = v14;
    }
    return v13;
}
/* estimate_all: matrices by reference (row-major), owned by the caller
 *   v1  double[2000][200]   in    read only
 *   v2  double[2000][2000]  in    read only
 *   v3  size_t              in
 *   v4  int8_t *            work  workspace (bytes)
 *   v5  int64_t *           info  -1: query, 0: allocate, >0: size of work
 * returns info:
 *     0  success
 *     1  call of gls returned info != 0
 *    -5  v5: workspace too small
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
estimate_all(double *v1, double *v2, size_t v3, int8_t *v4, int64_t *v5)
{
    int64_t v6 = 0;
    int64_t v7 = -64;
    int64_t v8 = 63;
    int64_t v9 = -1;
    size_t v10 = 0;
    int64_t v11 = v5[v10];
    int64_t v12 = (int64_t)v10;
    double *v13 = (double *)(v4 + v12);
    int64_t v14 = (int64_t)v10;
    double *v15 = (double *)(v4 + v14);
    int64_t v16 = (int64_t)v10;
    double *v17 = (double *)(v4 + v16);
    int64_t v18 = (int64_t)v10;
    double *v19 = (double *)(v4 + v18);
    v5[v10] = v9;
    int64_t v20 = gls(v13, v15, v17, v19, v4, v5);
    int64_t v21;
    if (v20 == v6) {
        int64_t v22 = v5[v10];
        uint64_t v23 = (uint64_t)v22;
        uint64_t v24 = (uint64_t)v8;
        int64_t v25 = (int64_t)(v23 + v24);
        uint64_t v26 = (uint64_t)v25;
        uint64_t v27 = (uint64_t)v7;
        int64_t v28 = (int64_t)(v26 & v27);
        uint64_t v29 = (uint64_t)v28;
        uint64_t v30 = (uint64_t)17600;
        int64_t v31 = (int64_t)(v29 + v30);
        v5[v10] = v9;
        postprocess(v10, v19, v4, v5);
        int64_t v32 = v5[v10];
        uint64_t v33 = (uint64_t)v32;
        uint64_t v34 = (uint64_t)v8;
        int64_t v35 = (int64_t)(v33 + v34);
        uint64_t v36 = (uint64_t)v35;
        uint64_t v37 = (uint64_t)v7;
```

```
        int64_t v38 = (int64_t)(v36 & v37);
        uint64_t v39 = (uint64_t)v31;
        uint64_t v40 = (uint64_t)v38;
        int64_t v41 = (int64_t)(v39 + v40);
        v5[v10] = v11;
        int64_t v42;
        if (v11 == v9) {
            v5[v10] = v41;
            v42 = v6;
        } else {
            int64_t v43;
            if (v11 > v6 & v11 < v41) {
                v43 = -5;
            } else {
                bool v44 = v11 == v6;
                int8_t *v45;
                if (v44) {
                    ptrdiff_t v46 = (ptrdiff_t)v41;
                    size_t v47 = (size_t)v46;
                    void *v48 = aligned_alloc(64, v47);
                    v45 = (int8_t *)v48;
                } else {
                    v45 = v4;
                }
                int8_t *v49 = v45;
                int64_t v50 = (int64_t)v10;
                double *v51 = (double *)(v49 + v50);
                int64_t v52 = (int64_t)16000;
                double *v53 = (double *)(v49 + v52);
                int64_t v54;
                v54 = v6;
                for (size_t i55 = v10; i55 < v3; i55 += 1) {
                    int64_t v56 = v54;
                    int64_t v57;
                    if (v56 == v6) {
                        observe(i55, v51);
                        int64_t v58 = (int64_t)17600;
                        v5[v10] = v22;
                        bool v59 =
                            (bool)(gls(v1, v2, v51, v53, v49 + v58, v5) != v6);
                        uint64_t v60 = (uint64_t)v59;
                        int64_t v61 = (int64_t)v60;
                        if (v61 == v6) {
                            ptrdiff_t v62 = (ptrdiff_t)v31;
                            size_t v63 = (size_t)v62;
                            int64_t v64 = (int64_t)v63;
                            int8_t *v65 = v49 + v64;
                            v5[v10] = v32;
                            postprocess(i55, v53, v65, v5);
                        }
                        v57 = v61;
                    } else {
                        v57 = v56;
                    }
                    v54 = v57;
                }
                int64_t v66 = v54;
                if (v66 == v6) {
                    v5[v10] = v11;
                }
                if (v44) {
                    free(v49);
                }
                v43 = v66;
            }
            v42 = v43;
        }
        v21 = v42;
    } else {
        v21 = v20;
    }
    return v21;
}
```

A.7 `examples/cases/generated/cases.cpp`

The kernel selection cases of `examples/cases/cases.mlir`.

```cpp
#include <cstddef>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <ulmblas/lapack/getrf.h>
#include <ulmblas/lapack/getrs.h>
#include <ulmblas/lapack/potrf.h>
#include <ulmblas/lapack/trtrs.h>
#include <ulmblas/ulmblas.h>
static void
ulm_lam_copy(void *dst, const void *src, std::size_t bytes)
{
#ifdef _OPENMP
    const std::size_t chunk = std::size_t(1) << 20;
    if (bytes >= 8 * chunk) {
        std::ptrdiff_t chunks = (bytes + chunk - 1) / chunk;
#pragma omp parallel for schedule(static)
        for (std::ptrdiff_t c = 0; c < chunks; ++c) {
            std::size_t off = c * chunk;
            std::size_t len = bytes - off < chunk ? bytes - off : chunk;
            std::memcpy(static_cast<char *>(dst) + off,
                        static_cast<const char *>(src) + off, len);
        }
        return;
    }
#endif
    std::memcpy(dst, src, bytes);
}

/* lu: matrices by reference (row-major), owned by the caller
 *   v1  double[64][64]  in    read only
 *   v2  double[64][32]  in    read only
 *   v3  double[64][32]  out   written
 *   v4  int8_t *        work  workspace (bytes)
 *   v5  int64_t *       info  -1: query, 0: allocate, >0: size of work
 * returns info:
 *     0  success
 *     1  LU factorisation of v1: singular
 *    -5  v5: workspace too small
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
lu(double *v1, double *v2, double *v3, int8_t *v4, int64_t *v5)
{
    int64_t v6 = 33024;
    int64_t v7 = 0;
    size_t v8 = 0;
    int64_t v9 = v5[v8];
    int64_t v10;
    if (v9 == -1) {
        v5[v8] = v6;
        v10 = v7;
    } else {
        int64_t v11;
        if (v9 > v7 & v9 < v6) {
            v11 = -5;
        } else {
            bool v12 = v9 == v7;
            int8_t *v13;
            if (v12) {
                void *v14 = aligned_alloc(64, 33024);
                v13 = (int8_t *)v14;
            } else {
                v13 = v4;
            }
            int8_t *v15 = v13;
            int64_t v16 = (int64_t)v8;
            int32_t *v17 = (int32_t *)(v15 + v16);
```

```
            int64_t v18 = (int64_t)256;
            double *v19 = (double *)(v15 + v18);
            ulm_lam_copy(v19, v1, 32768);
            int64_t v20 =
                ulmBLAS::getrf<std::ptrdiff_t>(64, 64, v19, 64, 1, v17, 1);
            bool v21 = (bool)(v20 != v7);
            uint64_t v22 = (uint64_t)v21;
            int64_t v23 = (int64_t)v22;
            if (v23 == v7) {
                ulm_lam_copy(v3, v2, 16384);
                ulmBLAS::getrs<std::ptrdiff_t>(false, false, 64, 32, v19, 64, 1,
                                               v17, 1, v3, 32, 1);
            }
            if (v12) {
                free(v15);
            }
            v11 = v23;
        }
        v10 = v11;
    }
    return v10;
}
/* lu_t: matrices by reference (row-major), owned by the caller
 *   v1  double[64][64]  in    read only
 *   v2  double[64][32]  in    read only
 *   v3  double[64][32]  out   written
 *   v4  int8_t *        work  workspace (bytes)
 *   v5  int64_t *       info  -1: query, 0: allocate, >0: size of work
 * returns info:
 *     0  success
 *     1  LU factorisation of v1: singular
 *    -5  v5: workspace too small
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
lu_t(double *v1, double *v2, double *v3, int8_t *v4, int64_t *v5)
{
    int64_t v6 = 33024;
    int64_t v7 = 0;
    size_t v8 = 0;
    int64_t v9 = v5[v8];
    int64_t v10;
    if (v9 == -1) {
        v5[v8] = v6;
        v10 = v7;
    } else {
        int64_t v11;
        if (v9 > v7 & v9 < v6) {
            v11 = -5;
        } else {
            bool v12 = v9 == v7;
            int8_t *v13;
            if (v12) {
                void *v14 = aligned_alloc(64, 33024);
                v13 = (int8_t *)v14;
            } else {
                v13 = v4;
            }
            int8_t *v15 = v13;
            int64_t v16 = (int64_t)v8;
            int32_t *v17 = (int32_t *)(v15 + v16);
            int64_t v18 = (int64_t)256;
            double *v19 = (double *)(v15 + v18);
            ulm_lam_copy(v19, v1, 32768);
            int64_t v20 =
                ulmBLAS::getrf<std::ptrdiff_t>(64, 64, v19, 64, 1, v17, 1);
            bool v21 = (bool)(v20 != v7);
            uint64_t v22 = (uint64_t)v21;
            int64_t v23 = (int64_t)v22;
            if (v23 == v7) {
                ulm_lam_copy(v3, v2, 16384);
                ulmBLAS::getrs<std::ptrdiff_t>(true, false, 64, 32, v19, 64, 1,
                                               v17, 1, v3, 32, 1);
```

```cpp
            }
            if (v12) {
                free(v15);
            }
            v11 = v23;
        }
        v10 = v11;
    }
    return v10;
}
/* upper: matrices by reference (row-major), owned by the caller
 *   v1  double[64][64]  in   read only
 *   v2  double[64][32]  in   read only
 *   v3  double[64][32]  out  written
 * returns info:
 *    0  success
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
upper(double *v1, double *v2, double *v3)
{
    ulm_lam_copy(v3, v2, 16384);
    (void)ulmBLAS::trtrs<std::ptrdiff_t>(false, false, false, false, 64, 32, v1,
                                         64, 1, v3, 32, 1);
    return 0;
}
/* right: matrices by reference (row-major), owned by the caller
 *   v1  double[32][64]  in   read only
 *   v2  double[64][64]  in   read only
 *   v3  double[32][64]  out  written
 * returns info:
 *    0  success
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
right(double *v1, double *v2, double *v3)
{
    ulm_lam_copy(v3, v1, 16384);
    (void)ulmBLAS::trtrs<std::ptrdiff_t>(false, false, false, false, 64, 32, v2,
                                         1, 64, v3, 1, 64);
    return 0;
}
/* right_t: matrices by reference (row-major), owned by the caller
 *   v1  double[32][64]  in   read only
 *   v2  double[64][64]  in   read only
 *   v3  double[32][64]  out  written
 * returns info:
 *    0  success
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
right_t(double *v1, double *v2, double *v3)
{
    ulm_lam_copy(v3, v1, 16384);
    (void)ulmBLAS::trtrs<std::ptrdiff_t>(true, false, false, false, 64, 32, v2,
                                         64, 1, v3, 1, 64);
    return 0;
}
/* chol_vec: matrices by reference (row-major), owned by the caller
 *   v1  double[64][64]  in    read only
 *   v2  double[64][1]   in    read only
 *   v3  double[64][1]   out   written
 *   v4  int8_t *        work  workspace (bytes)
 *   v5  int64_t *       info  -1: query, 0: allocate, >0: size of work
 * returns info:
 *     0  success
 *     1  Cholesky factorisation of v1: not positive definite
 *    -5  v5: workspace too small
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
chol_vec(double *v1, double *v2, double *v3, int8_t *v4, int64_t *v5)
{
```

```cpp
    int64_t v6 = 32768;
    int64_t v7 = 0;
    size_t v8 = 0;
    int64_t v9 = v5[v8];
    int64_t v10;
    if (v9 == -1) {
        v5[v8] = v6;
        v10 = v7;
    } else {
        int64_t v11;
        if (v9 > v7 & v9 < v6) {
            v11 = -5;
        } else {
            bool v12 = v9 == v7;
            int8_t *v13;
            if (v12) {
                void *v14 = aligned_alloc(64, 32768);
                v13 = (int8_t *)v14;
            } else {
                v13 = v4;
            }
            int8_t *v15 = v13;
            int64_t v16 = (int64_t)v8;
            double *v17 = (double *)(v15 + v16);
            ulm_lam_copy(v17, v1, 32768);
            int64_t v18 = ulmBLAS::potrf<std::ptrdiff_t>(true, 64, v17, 64, 1);
            bool v19 = (bool)(v18 != v7);
            uint64_t v20 = (uint64_t)v19;
            int64_t v21 = (int64_t)v20;
            if (v21 == v7) {
                ulm_lam_copy(v3, v2, 512);
                ulmBLAS::trlsv<std::ptrdiff_t>(64, false, v17, 64, 1, v3, 1);
                ulmBLAS::trusv<std::ptrdiff_t>(64, false, v17, 1, 64, v3, 1);
            }
            if (v12) {
                free(v15);
            }
            v11 = v21;
        }
        v10 = v11;
    }
    return v10;
}
/* sum: matrices by reference (row-major), owned by the caller
 *   v1  double[64][64]  in   read only
 *   v2  double[64][64]  in   read only
 *   v3  double[64][64]  in   read only
 *   v4  double[64][64]  out  written
 * returns info:
 *    0  success
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
sum(double *v1, double *v2, double *v3, double *v4)
{
    double v5 = 2.00000000000000000e+00;
    double v6 = -1.00000000000000000e+00;
    ulm_lam_copy(v4, v1, 32768);
    ulmBLAS::geaxpy<std::ptrdiff_t>(64, 64, v5, v2, 64, 1, v4, 64, 1);
    ulmBLAS::geaxpy<std::ptrdiff_t>(64, 64, v6, v3, 64, 1, v4, 64, 1);
    return 0;
}
/* mixed: matrices by reference (row-major), owned by the caller
 *   v1  double[64][64]  in    read only
 *   v2  double[64][64]  in    read only
 *   v3  double[64][32]  in    read only
 *   v4  double[64][32]  in    read only
 *   v5  double[64][32]  out   written
 *   v6  int8_t *        work  workspace (bytes)
 *   v7  int64_t *       info  -1: query, 0: allocate, >0: size of work
 * returns info:
 *     0  success
 *     1  LU factorisation of v1: singular
```

```
 *    -7  v7: workspace too small
 * (info != 0: the outputs are unspecified)
 */
extern "C" int64_t
mixed(double *v1, double *v2, double *v3, double *v4, double *v5, int8_t *v6,
      int64_t *v7)
{
    int64_t v8 = 49408;
    double v9 = 1.00000000000000000e+00;
    double v10 = 0.0e+00;
    int64_t v11 = 0;
    size_t v12 = 0;
    int64_t v13 = v7[v12];
    int64_t v14;
    if (v13 == -1) {
        v7[v12] = v8;
        v14 = v11;
    } else {
        int64_t v15;
        if (v13 > v11 & v13 < v8) {
            v15 = -7;
        } else {
            bool v16 = v13 == v11;
            int8_t *v17;
            if (v16) {
                void *v18 = aligned_alloc(64, 49408);
                v17 = (int8_t *)v18;
            } else {
                v17 = v6;
            }
            int8_t *v19 = v17;
            int64_t v20 = (int64_t)v12;
            int32_t *v21 = (int32_t *)(v19 + v20);
            int64_t v22 = (int64_t)256;
            double *v23 = (double *)(v19 + v22);
            ulm_lam_copy(v23, v1, 32768);
            int64_t v24 =
                ulmBLAS::getrf<std::ptrdiff_t>(64, 64, v23, 64, 1, v21, 1);
            bool v25 = (bool)(v24 != v11);
            uint64_t v26 = (uint64_t)v25;
            int64_t v27 = (int64_t)v26;
            if (v27 == v11) {
                int64_t v28 = (int64_t)33024;
                double *v29 = (double *)(v19 + v28);
                ulmBLAS::gemm<std::ptrdiff_t>(64, 32, 64, v9, false, v2, 64, 1,
                                              false, v3, 32, 1, v10, v29, 32,
                                              1);
                ulmBLAS::getrs<std::ptrdiff_t>(false, false, 64, 32, v23, 64, 1,
                                               v21, 1, v29, 32, 1);
                ulm_lam_copy(v5, v4, 16384);
                ulmBLAS::geaxpy<std::ptrdiff_t>(64, 32, v9, v29, 32, 1, v5, 32,
                                                1);
            }
            if (v16) {
                free(v19);
            }
            v15 = v27;
        }
        v14 = v15;
    }
    return v14;
}
```

References

[1] C. Psarras, H. Barthels, P. Bientinesi. The Linear Algebra Mapping Problem. Current state of linear algebra languages and libraries. *ACM TOMS*, 2022. doi:10.1145/3549935.

[2] H. Barthels, C. Psarras, P. Bientinesi. Linnea: Automatic generation of efficient linear algebra programs. *ACM TOMS*, 2021.

[3] H. Barthels. *Linnea: A compiler for mapping linear algebra problems onto high-performance kernel libraries*. PhD thesis, RWTH Aachen, 2022. doi:10.18154/RWTH-2023-01193.

[4] H. Barthels, M. Copik, P. Bientinesi. The generalized matrix chain algorithm. *CGO*, 2018.

[5] M. Krebber, H. Barthels. MatchPy: Pattern matching in Python. *JOSS*, 2018.

[6] A. Sankaran, N. A. Alashti, C. Psarras, P. Bientinesi. Benchmarking the linear algebra awareness of TensorFlow and PyTorch. arXiv:2202.09888, 2022.

[7] L. Chelini, H. Barthels, P. Bientinesi, M. Copik, T. Grosser, D. G. Spampinato. MOM: Matrix operations in MLIR. arXiv:2208.10391, 2022.

[8] C. Lattner et al. MLIR: Scaling compiler infrastructure for domain specific computation. *CGO*, 2021.

[9] M. Willsey et al. egg: Fast and extensible equality saturation. *POPL*, 2021.

[10] Y. Zhang et al. Better together: Unifying Datalog and equality saturation. *PLDI*, 2023.

[11] Y. R. Wang, S. Hutchison, J. Leang, B. Howe, D. Suciu. SPORES: Sum-product optimization via relational equality saturation for large scale linear algebra. *PVLDB*, 2020.

[12] eqsat: An equality saturation dialect for non-destructive rewriting. *EGRAPHS*, 2025. arXiv:2505.09363.

[13] DialEgg: Dialect-agnostic MLIR optimizer using equality saturation with egglog. *CGO*, 2025. doi:10.1145/3696443.3708957.

[14] M. Lehn, A. Stippler, K. Urban. FLENS – A flexible library for efficient numerical solutions. *Proceedings of Equadiff 11*, 2005. `https://www.dml.cz/bitstream/handle/10338.dmlcz/700445/Equadiff_11-2005-2_55.pdf`.

[15] M. Lehn. FLENS: Flexible Library for Efficient Numerical Solutions. `https://github.com/michael-lehn/FLENS`.

[16] M. Lehn. ulmBLAS: a BLAS implementation in C++. `https://github.com/michael-lehn/ulmBLAS`.

[17] LAPIS: A performance portable, high productivity compiler framework. 2025. arXiv:2509.25605.

File	Content
`include/ulm_lam/UlmLamBase.td`	the `ulm_lam` and `ulm_lamk` dialects
`include/ulm_lam/UlmLamAttrs.td`	property encoding `#ulm_lam.props`, kernel flags
`include/ulm_lam/UlmLamInterfaces.td`	property inference and cost interfaces
`include/ulm_lam/UlmLamOps.td`	`ulm_lam` operations
`include/ulm_lam/UlmLamKernelOps.td`	`ulm_lamk` operations (BLAS/LAPACK)
`include/ulm_lam/UlmLamPasses.td`	passes of the pipeline
`include/ulm_lam/UlmLamKernels.td`	declarative kernel catalogue
`examples/gls/0[1-4]-*.mlir`	the worked example: design stages 1–4
`examples/gls/run.sh`, `driver.cc`	from stage 2 to C++: compile, run, verify
`examples/gls/gls_inout.mlir`	GLS with M, y declared `inout`
`examples/cases/`	inverses, solves and sums, checked numerically
`examples/kalman/`	formula (1) in a loop: changing y; covariance updated in place (Section 10)
`examples/compose/`	workspaces of functions that call functions with workspaces
`examples/*/generated/`	the C++ generated from each example (Section A)
`examples/loop/`	temporaries in loops (Section 8.1)
`lib/UlmLamResultsToOutParams.cpp`	implemented pass: matrices by reference
`lib/ConvertUlmLamkToUlmBLAS.cpp`	lowering to C++ calling ulmBLAS
`lib/UlmLamWorkspace.cpp`	temporaries from a caller-provided workspace
`examples/workspace.h`	the workspace convention for the C++ drivers
`python/ulm/lam/`	the Python frontend and runtime (Section 11)
`python/examples/`, `python/tests/`	Python example and unit tests
`scripts/ulm-lam-compile.sh`	the pipeline: `ulm_lam` ops to C++ (all stages)
`scripts/build-plugin.sh`	build the `mlir-opt` pass plugin
`scripts/check-ods.sh`	run all `mlir-tblgen` backends
`scripts/check-stages.sh`	stages 1–4 parse and verify; stage 3 bufferizes as stage 4
`lib/UlmLamDialect.{h,cpp}`, `UlmLamkDialect.cpp`	the dialects: inference, verifiers, patterns, costs, bufferization
`lib/UlmLamPasses.cpp`	normal form, property inference, kernel selection, property stripping
`lib/Plugin.cpp`	`mlir-opt` dialect and pass plugin entry points
`doc/`	this document (LuaLaTeX)

Table 1: Repository contents. Toolchain used: Homebrew LLVM/MLIR 23.1.4, ulmBLAS (commit c2273e7, November 2026).

```
def UlmLam_PropNone       : I32BitEnumAttrCaseNone<"general">;
def UlmLam_PropSquare     : I32BitEnumAttrCaseBit<"square",       0>;
def UlmLam_PropSymmetric  : I32BitEnumAttrCaseBit<"symmetric",    1>;
def UlmLam_PropSPSD       : I32BitEnumAttrCaseBit<"spsd",         2>;
def UlmLam_PropSPD        : I32BitEnumAttrCaseBit<"spd",          3>;
def UlmLam_PropLowerTri   : I32BitEnumAttrCaseBit<"lower_tri",    4>;
def UlmLam_PropUpperTri   : I32BitEnumAttrCaseBit<"upper_tri",    5>;
def UlmLam_PropUnitDiag   : I32BitEnumAttrCaseBit<"unit_diag",    6>;
def UlmLam_PropIdentity   : I32BitEnumAttrCaseBit<"identity",     7>;
def UlmLam_PropZero       : I32BitEnumAttrCaseBit<"zero",         8>;
def UlmLam_PropOrthogonal : I32BitEnumAttrCaseBit<"orthogonal",   9>;
def UlmLam_PropPermutation: I32BitEnumAttrCaseBit<"permutation", 10>;
def UlmLam_PropNonSingular: I32BitEnumAttrCaseBit<"non_singular",11>;
def UlmLam_PropFullRank   : I32BitEnumAttrCaseBit<"full_rank",   12>;
def UlmLam_PropLowerStored: I32BitEnumAttrCaseBit<"lower_stored",13>;
def UlmLam_PropUpperStored: I32BitEnumAttrCaseBit<"upper_stored",14>;
def UlmLam_PropDiagonal   : I32BitEnumAttrCaseGroup<"diagonal",
                              [UlmLam_PropLowerTri, UlmLam_PropUpperTri]>;

def UlmLam_Properties : I32BitEnumAttr<"Properties", "matrix properties", [
    UlmLam_PropNone, UlmLam_PropSquare, UlmLam_PropSymmetric, UlmLam_PropSPSD,
    UlmLam_PropSPD, UlmLam_PropLowerTri, UlmLam_PropUpperTri, UlmLam_PropUnitDiag,
    UlmLam_PropIdentity, UlmLam_PropZero, UlmLam_PropOrthogonal,
    UlmLam_PropPermutation, UlmLam_PropNonSingular, UlmLam_PropFullRank,
    UlmLam_PropLowerStored, UlmLam_PropUpperStored, UlmLam_PropDiagonal]> {
  let separator = ", ";
  let cppNamespace = "::mlir::ulm_lam";
  let genSpecializedAttr = 0;
  let printBitEnumPrimaryGroups = 1;
}

// Used as a builtin tensor encoding:  tensor<?x?xf64, #ulm_lam.props<spd>>
def UlmLam_PropsAttr : EnumAttr<UlmLam_Dialect, UlmLam_Properties, "props",
    [DeclareAttrInterfaceMethods<VerifiableTensorEncoding>]> {
  let assemblyFormat = "`<` $value `>`";
  let description = [{
    Matrix properties attached to a rank-2 tensor.  `verifyEncoding` rejects
    contradictions (e.g. `zero` with `non_singular`) and properties that
    contradict a static shape (e.g. `square` on a 3x4 tensor).
  }];
}

//===----------------------------------------------------------------------===//
```

Figure 3: The property encoding (`UlmLamAttrs.td`).

Expression	Inferred properties (beyond closure)
A^{T}	lower ↔ upper swapped; all symmetric, definite, diagonal, orthogonal, rank properties preserved
A^{-1}	requires non-singular; triangular, unit diagonal, diagonal, symmetric, spd, orthogonal, permutation preserved
$A_1 \cdots A_k$	lower (upper) if all are; unit diagonal, diagonal, orthogonal, permutation, non-singular if all are; zero if any is
$A^{\mathsf{T}}A$ / AA^{T}	spsd; spd if A has full column (row) rank
$A^{\mathsf{T}}PA$	spd if P spd and A full column rank; spsd if P spsd
NA, AN	full rank if A is and N is non-singular
$A_1 + \cdots + A_k$	symmetric / spsd / triangular / diagonal if all are; spd if all spsd and one spd
αA	structure preserved; spd, spsd only for constant $\alpha > 0$; non-singular for constant $\alpha \neq 0$
`ulm_lamk` results	the same functions, through `PropertyInferenceOpInterface`: e.g. `trsm` result $L^{-1}X$ is full rank if X is

Table 2: Local property inference rules (selection).

Op	Semantics
`ulm_lam.mul`	n-ary product $A_1 \cdots A_k$; associative, no parenthesisation
`ulm_lam.add`	n-ary sum; associative and commutative (`Commutative` trait)
`ulm_lam.scale`	αA; subtraction is `add(A, scale(-1, B))`
`ulm_lam.transpose`	A^{T}
`ulm_lam.inverse`	A^{-1}; verifier requires square; never computed unless requested
`ulm_lam.identity`	identity matrix (with dynamic sizes as operands)
`ulm_lam.assume`	add properties (trusted, or `checked` at run time)

Table 3: `ulm_lam` operations. All are `Pure`.

```
def UlmLam_MulOp : UlmLam_Op<"mul"> {
  let summary = "n-ary matrix product (associative, not commutative)";
  let description = [{
    `ulm_lam.mul %A, %B, %C` is the product A*B*C without a parenthesisation.
    Choosing the parenthesisation is part of kernel selection (generalised
    matrix chain problem).  Nested products are flattened by the
    canonicalizer; the verifier checks inner dimensions.

    ```mlir
    %S = ulm_lam.mul %Xt, %Mi, %X
       : tensor<200x2000xf64>, tensor<2000x2000xf64, #spd>,
         tensor<2000x200xf64, #fr> -> tensor<200x200xf64, #spd>
    ```
  }];
  let arguments = (ins Variadic<UlmLam_Matrix>:$factors);
  let results = (outs UlmLam_Matrix:$result);
  let assemblyFormat =
      "$factors attr-dict `:` type($factors) `->` type($result)";
  let hasVerifier = 1;
  let hasCanonicalizer = 1;
}
```

Figure 4: `ulm_lam.mul` in ODS. The common base class (not shown) adds `Pure`, `InferTypeOpInterface` with relaxed compatibility, and `PropertyInferenceOpInterface`.

Op	ulmBLAS	Semantics	FLOPs
`gemm`	`gemm`	$C \leftarrow \alpha\,\mathrm{op}(A)\,\mathrm{op}(B) + \beta C$	$2mnk$
`symm`	`sylmm`	$C \leftarrow \alpha SB + \beta C$, S symmetric	$2m^2n$
`trmm`	`trlmm`/`trumm`	$B \leftarrow \alpha\,\mathrm{op}(L)B$	m^2n
`trsm`	`trlsm`/`trusm`	$B \leftarrow \alpha\,\mathrm{op}(L)^{-1}B$	m^2n
`syrk`	`sylrk`	$C \leftarrow \alpha A^{\mathsf{T}}A + \beta C$ (one triangle)	n^2k
`gemv`	`gemv`	$y \leftarrow \alpha\,\mathrm{op}(A)x + \beta y$	$2mn$
`trsv`	`trlsv`/`trusv`	$x \leftarrow \mathrm{op}(L)^{-1}x$	n^2
`potrf`	`potrf`	$A = LL^{\mathsf{T}}$, in place	$n^3/3$
`getrf`	`getrf`	$PA = LU$ (lower–upper), in place	$2n^3/3$
`getrs`	`getrs`	$B \leftarrow \mathrm{op}(A)^{-1}B$ from LU	$2n^2k$
`geaxpy`	`geaxpy`	$Y \leftarrow \alpha X + Y$ (matrices)	$2mn$
`gescal`	`gescal`	$X \leftarrow \alpha X$ (matrices)	mn
`call`	–	external function updating its `outs` in place	–

Table 4: `ulm_lamk` operations and the ulmBLAS routines they are lowered to. The BLAS-like operations keep the familiar `trans` and `side` attributes; in the lowering they become strides (Section 8.1). `trmm` is defined but not yet produced by kernel selection. Not listed: `symv` (`sylmv`) and `sycomplete` (a helper in the generated code), see Section 6.

```
def UlmLamk_TrsmOp : UlmLamk_Op<"trsm", "B", [ScalarOf<"alpha", "A">]> {
  let summary = "B := alpha * op(A)^-1 * B  (A triangular, in place)";
  let arguments = (ins UlmLamk_SideAttr:$side, UlmLamk_UploAttr:$uplo,
                       UlmLamk_TransAttr:$trans, UlmLamk_DiagAttr:$diag,
                       UlmLamk_Float:$alpha, UlmLamk_Shaped:$A, UlmLamk_Shaped:$B);
  let assemblyFormat = [{
    `side` `(` $side `)` `uplo` `(` $uplo `)` `trans` `(` $trans `)`
    `diag` `(` $diag `)` `alpha` `(` $alpha `)`
    `ins` `(` $A `:` type($A) `)` `outs` `(` $B `:` type($B) `)`
    attr-dict (`->` type($result)^)?
  }];
}
```

Figure 5: `ulm_lamk.trsm` in ODS. The base class `UlmLamk_Op` adds `DestinationStyleOpInterface` (with `B` as init), memory effects, property inference and the cost interface.

```
// Triangular solves with a matrix right-hand side.
def : Kernel<(mul (inv $L), $B), [(has $L, LowerTri)],
             Emit<"trsm", [Left, Lower, N, NonUnit]>, (ins $L, $B),
             "rows(L) * rows(L) * cols(B)", "B">;
def : Kernel<(mul (trans (inv $L)), $B), [(has $L, LowerTri)],
             Emit<"trsm", [Left, Lower, T, NonUnit]>, (ins $L, $B),
             "rows(L) * rows(L) * cols(B)", "B">;
def : Kernel<(mul (inv $U), $B), [(has $U, UpperTri)],
             Emit<"trsm", [Left, Upper, N, NonUnit]>, (ins $U, $B),
             "rows(U) * rows(U) * cols(B)", "B">;
def : Kernel<(mul $B, (inv $L)), [(has $L, LowerTri)],
             Emit<"trsm", [Right, Lower, N, NonUnit]>, (ins $L, $B),
             "rows(B) * rows(L) * rows(L)", "B">;
```

```
def Cholesky : Factorization<"cholesky", [(has $A, SPD)],
    (mul $L, (trans $L)), [(has $L, LowerTri, NonSingular)],
    Emit<"potrf", [Lower]>, (ins $A), "rows(A) * rows(A) * rows(A) / 3">;
```

Figure 6: Excerpt of the kernel catalogue (`UlmLamKernels.td`).

```
int64_t estimate_all(double *X, double *M, size_t n,
                     int8_t *work, int64_t *workspace_info)
{
    int64_t given = *workspace_info;
    *workspace_info = -1;                          // query the callees
    gls(dummy, dummy, dummy, dummy, work, workspace_info);
    int64_t q1 = *workspace_info;
    *workspace_info = -1;
    postprocess(0, dummy, work, workspace_info);
    int64_t q2 = *workspace_info;
    *workspace_info = given;
    int64_t required = 17600 + up64(q1) + up64(q2);
    if (given == -1) { *workspace_info = required; return 0; }
    if (given > 0 && given < required) return -5;  // info: argument 5
    int8_t *ws = given == 0 ? aligned_alloc(64, required) : work;
    double *y = (double *)(ws + 0), *b = (double *)(ws + 16000);
    int64_t info = 0;
    for (size_t i = 0; i < n; ++i) {
        if (info != 0) continue;   // a failed step: the others do nothing
        observe(i, y);
        *workspace_info = q1;
        if (gls(X, M, y, b, ws + 17600, workspace_info) != 0) { info = 1; continue; }
        *workspace_info = q2;  postprocess(i, b, ws + 17600 + up64(q1), workspace_info);
    }
    *workspace_info = given;
    if (given == 0) free(ws);
    return info;
}
```

Figure 7: The structure of the generated `estimate_all` (`examples/compose/compose.mlir`), written out by hand with readable names; the generated code is in Section A. `dummy` stands for pointers that a query does not touch, `up64` for rounding up to a multiple of 64.

`ulm_lamk` op	ulmBLAS call (row-major $m \times n$ storage: strides $(n, 1)$)
`gemm`	`gemm(m, n, k, alpha, false, A, incRowA, incColA, false, B, ..., beta, C, ...)`
`gemv`	`gemv(m, n, alpha, A, incRowA, incColA, x, incX, beta, y, incY)`
`trsm`	`trlsm` / `trusm(m, n, alpha, false, unitDiag, A, ..., B, ...)`
`trsv`	`trlsv` / `trusv(n, unitDiag, A, incRowA, incColA, x, incX)`
`geaxpy`	`geaxpy(m, n, alpha, X, incRowX, incColX, Y, incRowY, incColY)`
`gescal`	`gescal(m, n, alpha, X, incRowX, incColX)`
`potrf`	`info = potrf(lower, n, A, incRowA, incColA)`
`getrf`	`info = getrf(m, n, A, incRowA, incColA, piv, incPiv)` (0-based pivots)
`getrs`	`getrs(trans, false, n, nrhs, LU, ..., piv, incPiv, B, ...)`

Table 5: The ulmBLAS routines used by `convert-ulm-lamk-to-ulmblas`. `trans` selects the strides; `trans` and `uplo` together select `trl*` or `tru*`.

```
// Matrix operations inside a loop, as the frontend emits them:
//
//   for i = 0, ..., n-1:   B = foo(i);   X += A * B * C
//
// X is passed by reference: the result is marked {ulm_lam.inout = 2}, i.e. it
// is the updated value of argument 2.  B = foo(i) calls an external
// function that returns a matrix.
//
// The pipeline (examples/loop/run-c.sh):
//   ulm-lam-normalize, ulm-lam-infer-properties
//   ulm-lam-select-kernels       A B C by the matrix chain DP; X + A B C fused
//                              into the last gemm (beta = 1, outs = X)
//   ulm-lam-strip-properties
//   ulm-lam-results-to-out-params  foo(i) -> foo(i, B), X by reference
//   one-shot-bufferize, buffer-loop-hoisting, buffer-deallocation-pipeline
//   convert-ulm-lamk-to-emitc    gemm -> cblas_dgemm
//   mlir-translate --mlir-to-cpp, clang-format

!m = tensor<500x500xf64>
func.func private @foo(index) -> !m

func.func @accumulate(%A: !m, %C: !m, %X0: !m, %n: index)
    -> (!m {ulm_lam.inout = 2 : i64}) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index
  %X = scf.for %i = %c0 to %n step %c1 iter_args(%X = %X0) -> !m {
    %B  = func.call @foo(%i) : (index) -> !m
    %P  = ulm_lam.mul %A, %B, %C : !m, !m, !m -> !m
    %Xn = ulm_lam.add %X, %P : !m, !m -> !m
    scf.yield %Xn : !m
  }
  return %X : !m
}
```

Figure 8: The loop of Section 8.1 as the frontend emits it (`examples/loop/accumulate.mlir`).

	Linnea (stages 3, 4)		ulm.lam, `strategy=gmc`	
Step	Kernel	FLOPs	Kernel	FLOPs
$M = LL^{\mathsf{T}}$	`potrf`	$2.67 \cdot 10^9$	`potrf`	$2.67 \cdot 10^9$
$W = L^{-1}X$	`trsm`	$8.00 \cdot 10^8$	`trsm`	$8.00 \cdot 10^8$
$S = W^{\mathsf{T}}W$	`syrk`	$8.00 \cdot 10^7$	`syrk`	$8.00 \cdot 10^7$
$S = KK^{\mathsf{T}}$	`potrf`	$2.67 \cdot 10^6$	`potrf`	$2.67 \cdot 10^6$
$c = W^{\mathsf{T}}L^{-1}y$	`trsv`, `gemv`	$4.80 \cdot 10^6$	`trsv`, `gemv`	$4.80 \cdot 10^6$
$b = K^{-\mathsf{T}}K^{-1}c$	2×`trsv`	$8.00 \cdot 10^4$	2×`trsv`	$8.00 \cdot 10^4$
total		$3.55 \cdot 10^9$		$3.55 \cdot 10^9$

Table 6: The derivation for Eq. (1), with $n = 2000$ and $m = 200$. Evaluating `inv(X'*inv(M)*X)*X'*inv(M)*y` literally, with `getrf`+`getri` ($2n^3$) for each inverse, costs $1.78 \cdot 10^{10}$ FLOPs.

```
module {
  func.func private @foo(index) -> tensor<500x500xf64>
  func.func @accumulate(%arg0: tensor<500x500xf64>, %arg1: tensor<500x500xf64>, %arg2: tensor<500x500xf64>
     , %arg3: index) -> (tensor<500x500xf64> {ulm_lam.inout = 2 : i64}) {
    %cst = arith.constant 0.000000e+00 : f64
    %cst_0 = arith.constant 1.000000e+00 : f64
    %c0 = arith.constant 0 : index
    %c1 = arith.constant 1 : index
    %0 = scf.for %arg4 = %c0 to %arg3 step %c1 iter_args(%arg5 = %arg2) -> (tensor<500x500xf64>) {
      %1 = func.call @foo(%arg4) : (index) -> tensor<500x500xf64>
      %2 = tensor.empty() : tensor<500x500xf64>
      %3 = ulm_lamk.gemm trans_a(none) trans_b(none) alpha(%cst_0) beta(%cst) ins(%arg0, %1 : tensor<500
     x500xf64>, tensor<500x500xf64>) outs(%2 : tensor<500x500xf64>) -> tensor<500x500xf64>
      %4 = ulm_lamk.gemm trans_a(none) trans_b(none) alpha(%cst_0) beta(%cst_0) ins(%3, %arg1 : tensor<500
     x500xf64>, tensor<500x500xf64>) outs(%arg5 : tensor<500x500xf64>) -> tensor<500x500xf64>
      scf.yield %4 : tensor<500x500xf64>
    }
    return %0 : tensor<500x500xf64>
  }
}
```

Figure 9: The loop of Fig. 8 after `ulm-lam-normalize`, `ulm-lam-infer-properties`, `ulm-lam-select-kernels` and `ulm-lam-strip-properties`. The matrix chain chose $(AB)C$: the first gemm writes $T = AB$ into a new tensor ($\beta = 0$, `%cst`), the second accumulates TC into the loop-carried X (`%arg5`, $\beta = 1$, `%cst_0`).

Kernel sequence chosen by ulm.lam	29.6 ms
Explicit inverses (same library)	110.3 ms
Speedup	3.7×
Relative residual $\|X^{\mathsf{T}}M^{-1}(Xb - y)\| / \|X^{\mathsf{T}}M^{-1}y\|$	$1.2 \cdot 10^{-15}$
$\|b - b_{\text{inv}}\| / \|b\|$	$2.7 \cdot 10^{-15}$

Table 7: The generated GLS code on an Apple M5 (10 cores), ulmBLAS with `-mcpu=native`, best of 5 runs after warm-up.

	before the loop	per step	20 steps	
ulm.lam	15.3 ms	0.17 ms	18.6 ms	1×
ulm.lam without `loop-invariant-code-motion`	16.8 ms	6.13 ms	139.3 ms	7.5×
explicit inverses in every step	0.2 ms	81.6 ms	1631.2 ms	88×
explicit inverses, hoisted by hand	69.4 ms	0.21 ms	73.7 ms	4.0×
operator G, hoisted by hand	66.5 ms	0.24 ms	71.2 ms	3.8×
worst normal-equation residual over all steps				$1.0 \cdot 10^{-15}$
largest relative difference between the versions				$2.1 \cdot 10^{-15}$

Table 8: Formula (1) in a loop, $n = 2000$, $m = 200$, on an Apple M5, ulmBLAS with `-mcpu=native` and OpenMP. The time per step is measured over 200 steps (for ulm.lam it includes forming the operator P before the loop, which pays off from 24 steps on) and includes `observe`, which computes Hx_i plus noise; "20 steps" is the part before the loop plus 20 times the time per step.

```
module {
  func.func private @foo(index, memref<500x500xf64>)
  func.func @accumulate(%arg0: memref<500x500xf64> {ulm_lam.in}, %arg1: memref<500x500xf64> {ulm_lam.in}, %arg2: memref
     <500x500xf64> {ulm_lam.inout, ulm_lam.updated}, %arg3: index, %arg4: memref<?xi8> {ulm_lam.workspace}, %arg5:
     memref<1xi64> {ulm_lam.workspace}) -> i64 attributes {ulm_lam.info = []} {
    %c-6_i64 = arith.constant -6 : i64
    %c4000000_i64 = arith.constant 4000000 : i64
    %c2000000 = arith.constant 2000000 : index
    %c1 = arith.constant 1 : index
    %cst = arith.constant 1.000000e+00 : f64
    %cst_0 = arith.constant 0.000000e+00 : f64
    %c0_i64 = arith.constant 0 : i64
    %c-1_i64 = arith.constant -1 : i64
    %c0 = arith.constant 0 : index
    %0 = memref.load %arg5[%c0] : memref<1xi64>
    %1 = arith.cmpi eq, %0, %c-1_i64 : i64
    %2 = scf.if %1 -> (i64) {
      memref.store %c4000000_i64, %arg5[%c0] : memref<1xi64>
      scf.yield %c0_i64 : i64
    } else {
      %3 = arith.cmpi sgt, %0, %c0_i64 : i64
      %4 = arith.cmpi slt, %0, %c4000000_i64 : i64
      %5 = arith.andi %3, %4 : i1
      %6 = arith.select %5, %c-6_i64, %c0_i64 : i64
      scf.if %5 {
      } else {
        %7 = arith.cmpi eq, %0, %c0_i64 : i64
        %8 = scf.if %7 -> (memref<?xi8>) {
          %alloc = memref.alloc() {alignment = 64 : i64} : memref<4000000xi8>
          %cast = memref.cast %alloc : memref<4000000xi8> to memref<?xi8>
          scf.yield %cast : memref<?xi8>
        } else {
          scf.yield %arg4 : memref<?xi8>
        }
        %view = memref.view %8[%c0][] : memref<?xi8> to memref<500x500xf64>
        %view_1 = memref.view %8[%c2000000][] : memref<?xi8> to memref<500x500xf64>
        scf.for %arg6 = %c0 to %arg3 step %c1 {
          func.call @foo(%arg6, %view) : (index, memref<500x500xf64>) -> ()
          ulm_lamk.gemm trans_a(none) trans_b(none) alpha(%cst) beta(%cst_0) ins(%arg0, %view : memref<500x500xf64>,
     memref<500x500xf64>) outs(%view_1 : memref<500x500xf64>)
          ulm_lamk.gemm trans_a(none) trans_b(none) alpha(%cst) beta(%cst) ins(%view_1, %arg1 : memref<500x500xf64>,
     memref<500x500xf64>) outs(%arg2 : memref<500x500xf64>)
        }
        scf.if %7 {
          memref.dealloc %8 : memref<?xi8>
        }
      }
      scf.yield %6 : i64
    }
    return %2 : i64
  }
}
```

Figure 10: The loop of Fig. 9 after `ulm-lam-results-to-out-params`, One-Shot Bufferize, `buffer-loop-hoisting`, deallocation and `ulm-lam-workspace` (generated by `scripts/ulm-lam-compile.sh`). B (`%view`) and T (`%view_1`) are slots of the workspace, set up before the `scf.for`; the `scf.if` around the body implements the workspace query and check. `foo` writes B by reference, and X (`%arg2`) is updated in place.

```
accumulate(double *v1, double *v2, double *v3, size_t v4, int8_t *v5,
           int64_t *v6)
{
    int64_t v7 = 4000000;
    double v8 = 1.0000000000000000e+00;
    double v9 = 0.0e+00;
    int64_t v10 = 0;
    size_t v11 = 0;
    int64_t v12 = v6[v11];
    int64_t v13;
    if (v12 == -1) {
        v6[v11] = v7;
        v13 = v10;
    } else {
        bool v14 = v12 > v10 & v12 < v7;
        if (v14) {
        } else {
            bool v15 = v12 == v10;
            int8_t *v16;
            if (v15) {
                void *v17 = aligned_alloc(64, 4000000);
                v16 = (int8_t *)v17;
            } else {
                v16 = v5;
            }
            int8_t *v18 = v16;
            int64_t v19 = (int64_t)v11;
            double *v20 = (double *)(v18 + v19);
            int64_t v21 = (int64_t)2000000;
            double *v22 = (double *)(v18 + v21);
            for (size_t i23 = v11; i23 < v4; i23 += 1) {
                foo(i23, v20);
                ulmBLAS::gemm<std::ptrdiff_t>(500, 500, 500, v8, false, v1, 500,
                                              1, false, v20, 500, 1, v9, v22,
                                              500, 1);
                ulmBLAS::gemm<std::ptrdiff_t>(500, 500, 500, v8, false, v22,
                                              500, 1, false, v2, 500, 1, v8, v3,
                                              500, 1);
            }
            if (v15) {
                free(v18);
            }
        }
        v13 = v14 ? -6 : v10;
    }
    return v13;
}
```

Figure 11: The loop of Fig. 8 in C++, generated from the `ulm_lam` ops by kernel selection, `ulm-lam-results-to-out-params`, upstream bufferization, `convert-ulm-lamk-to-ulmblas`, `mlir-translate --mlir-to-cpp` and `clang-format`: the function `extern "C" int64_t accumulate(...)`, without the `#include` lines in front of it. B (`v18`) and T (`v20`) live in the workspace; `foo` writes B by reference; X (`v3`) is updated in place by the second `ulmBLAS::gemm` ($\beta = 1$).

```
module {
  func.func @gls(%arg0: tensor<2000x200xf64>, %arg1: tensor<2000x2000xf64>, %arg2: tensor<2000x1xf64>) ->
     tensor<200x1xf64> {
    %cst = arith.constant 0.000000e+00 : f64
    %cst_0 = arith.constant 1.000000e+00 : f64
    %result, %info = ulm_lamk.potrf uplo(lower) outs(%arg1 : tensor<2000x2000xf64>) -> tensor<2000
     x2000xf64>
    %0 = tensor.empty() : tensor<200x200xf64>
    %1 = ulm_lamk.trsm side(left) uplo(lower) trans(none) diag(non_unit) alpha(%cst_0) ins(%result :
     tensor<2000x2000xf64>) outs(%arg0 : tensor<2000x200xf64>) -> tensor<2000x200xf64>
    %2 = ulm_lamk.syrk uplo(lower) trans(trans) alpha(%cst_0) beta(%cst) ins(%1 : tensor<2000x200xf64>)
     outs(%0 : tensor<200x200xf64>) -> tensor<200x200xf64>
    %result_1, %info_2 = ulm_lamk.potrf uplo(lower) outs(%2 : tensor<200x200xf64>) -> tensor<200x200xf64>
    %3 = ulm_lamk.trsv uplo(lower) trans(none) diag(non_unit) ins(%result : tensor<2000x2000xf64>) outs(
     %arg2 : tensor<2000x1xf64>) -> tensor<2000x1xf64>
    %4 = tensor.empty() : tensor<200x1xf64>
    %5 = ulm_lamk.gemv trans(trans) alpha(%cst_0) beta(%cst) ins(%1, %3 : tensor<2000x200xf64>, tensor
     <2000x1xf64>) outs(%4 : tensor<200x1xf64>) -> tensor<200x1xf64>
    %6 = ulm_lamk.trsv uplo(lower) trans(none) diag(non_unit) ins(%result_1 : tensor<200x200xf64>) outs(%5
      : tensor<200x1xf64>) -> tensor<200x1xf64>
    %7 = ulm_lamk.trsv uplo(lower) trans(trans) diag(non_unit) ins(%result_1 : tensor<200x200xf64>) outs(
     %6 : tensor<200x1xf64>) -> tensor<200x1xf64>
    return %7 : tensor<200x1xf64>
  }
}
```

Figure 12: GLS after `ulm-lam-select-kernels`, generated from stage 2.

```
func.func private @observe(index) -> !y
func.func private @record(index, !b)

func.func @estimate(%H: !H, %R: !R, %n: index) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index
  scf.for %i = %c0 to %n step %c1 {
    %y  = func.call @observe(%i) : (index) -> !y
    %Ht = ulm_lam.transpose %H : !H -> !Ht
    %Ri = ulm_lam.inverse %R : !R -> !R
    %S  = ulm_lam.mul %Ht, %Ri, %H : !Ht, !R, !H -> !S
    %Si = ulm_lam.inverse %S : !S -> !S
    %b  = ulm_lam.mul %Si, %Ht, %Ri, %y : !S, !Ht, !R, !y -> !b
    func.call @record(%i, %b) : (index, !b) -> ()
  }
  return
}
```

Figure 14: Formula (1) in a loop (`examples/kalman/estimate.mlir`, without the header comment and the type aliases). `observe` delivers y_i, `record` consumes b_i.

```
                    for (size_t i40 = v10; i40 < v3; i40 += 1) {
                        observe(i40, v37);
                        if (v33) {
                            ulmBLAS::gemv<std::ptrdiff_t>(200, 2000, v7, v35,
                                                          2000, 1, v37, 1, v8,
                                                          v39, 1);
                        } else {
                            ulmBLAS::trlsv<std::ptrdiff_t>(2000, false, v19,
                                                           2000, 1, v37, 1);
                            ulmBLAS::gemv<std::ptrdiff_t>(
                                200, 2000, v7, v28, 1, 200, v37, 1, v8, v39, 1);
                            ulmBLAS::trlsv<std::ptrdiff_t>(200, false, v26, 200,
                                                           1, v39, 1);
                            ulmBLAS::trusv<std::ptrdiff_t>(200, false, v26, 1,
                                                           200, v39, 1);
                        }
                        record(i40, v39);
                    }
                }
                v24 = v30;
            } else {
                v24 = v23;
            }
            int64_t v41 = v24;
            if (v14) {
```

Figure 15: The loop body of the generated C++ (`scripts/ulm-lam-compile.sh` on Fig. 14). The factorisations, W and S are computed before the loop.

```
// update_covariance(i, R) updates R in place (the result is argument 1)
func.func private @update_covariance(index, !R) -> (!R {ulm_lam.inout = 1 : i64})
func.func private @observe(index) -> !y
func.func private @record(index, !b)

func.func @estimate_adaptive(%H: !H, %R0: !R, %n: index) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index
  %Rn = scf.for %i = %c0 to %n step %c1 iter_args(%R = %R0) -> !R {
    %Ri = func.call @update_covariance(%i, %R) : (index, !R) -> !R
    %y  = func.call @observe(%i) : (index) -> !y
    %Ht = ulm_lam.transpose %H : !H -> !Ht
    %Ri_inv = ulm_lam.inverse %Ri : !R -> !R
    %S  = ulm_lam.mul %Ht, %Ri_inv, %H : !Ht, !R, !H -> !S
    %Si = ulm_lam.inverse %S : !S -> !S
    %b  = ulm_lam.mul %Si, %Ht, %Ri_inv, %y : !S, !Ht, !R, !y -> !b
    func.call @record(%i, %b) : (index, !b) -> ()
    scf.yield %Ri : !R
  }
  return
}
```

Figure 16: The covariance updated in every step (`examples/kalman/adaptive.mlir`, without the header comment and the type aliases).

```
            for (size_t i31 = v10; i31 < v3; i31 += 1) {
                int64_t v32 = v30;
                int64_t v33;
                if (v32 == v9) {
                    update_covariance(i31, v19);
                    ulm_lam_copy(v21, v19, 32000000);
                    int64_t v34 = ulmBLAS::potrf<std::ptrdiff_t>(true, 2000,
                                                                 v21, 2000, 1);
                    bool v35 = (bool)(v34 != v9);
                    uint64_t v36 = (uint64_t)v35;
                    int64_t v37 = (int64_t)v36;
                    int64_t v38;
                    if (v37 == v9) {
                        observe(i31, v23);
                        ulm_lam_copy(v27, v1, 3200000);
                        (void)ulmBLAS::trtrs<std::ptrdiff_t>(
                            true, false, false, false, 2000, 200, v21, 2000, 1,
                            v27, 200, 1);
                        ulmBLAS::sylrk<std::ptrdiff_t>(200, 2000, v7, v27, 1,
                                                       200, v8, v25, 200, 1);
                        int64_t v39 = ulmBLAS::potrf<std::ptrdiff_t>(
                            true, 200, v25, 200, 1);
                        int64_t v40 = v39 != v9 ? 2 : v9;
                        if (v40 == v9) {
                            ulmBLAS::trlsv<std::ptrdiff_t>(2000, false, v21,
                                                           2000, 1, v23, 1);
                            ulmBLAS::gemv<std::ptrdiff_t>(
                                200, 2000, v7, v27, 1, 200, v23, 1, v8, v29, 1);
                            ulmBLAS::trlsv<std::ptrdiff_t>(200, false, v25, 200,
                                                           1, v29, 1);
                            ulmBLAS::trusv<std::ptrdiff_t>(200, false, v25, 1,
                                                           200, v29, 1);
                            record(i31, v29);
                        }
                        v38 = v40;
                    } else {
                        v38 = v37;
                    }
                    v33 = v38;
                } else {
                    v33 = v32;
                }
                v30 = v33;
            }
            int64_t v41 = v30;
            if (v14) {
```

Figure 17: The loop body generated from Fig. 16: an in-place update of R, two copies, two Cholesky factorisations, one `trlsm`, and the solves of Fig. 15.

```
observe = lam.extern("observe", [Index], Vector(n))
record = lam.extern("record", [Index, Vector(m)])


@lam.function
def estimate(H: Matrix(n, m, "full_rank"), R: Matrix(n, n, "spd"), steps: Index):
    with lam.loop(steps) as L:
        y = observe(L.index)
        b = (H.T @ R.inv @ H).inv @ H.T @ R.inv @ y
        record(L.index, b)
```

Figure 18: Formula (1) in a loop, in Python (from `python/examples/kalman.py`). `observe` and `record` are external functions, implemented in Python by callbacks.

```
update_covariance = lam.extern("update_covariance",
                               [Index, Matrix(n, n, "spd")],
                               Matrix(n, n, "spd"), updates=1)
observe = lam.extern("observe", [Index], Vector(n))
record = lam.extern("record", [Index, Vector(m)])


@lam.function
def estimate_adaptive(H: Matrix(n, m, "full_rank"), R0: Matrix(n, n, "spd"),
                      steps: Index):
    with lam.loop(steps, carry=[R0]) as L:
        (R,) = L.carried
        R = update_covariance(L.index, R)
        y = observe(L.index)
        b = (H.T @ R.inv @ H).inv @ H.T @ R.inv @ y
        record(L.index, b)
        L.next(R)
```

Figure 19: The adaptive filter in Python (from `python/examples/adaptive_kalman.py`): R is loop-carried and updated in place by an external function, a Python callback.

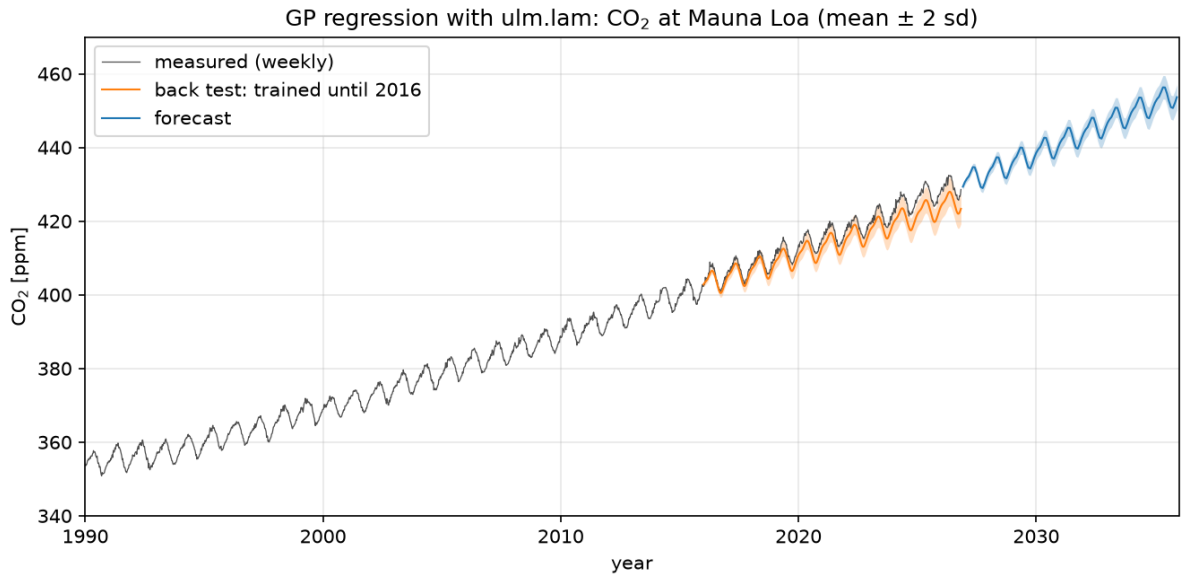


Figure 20: CO_2 at Mauna Loa: back test and forecast by `gp` (mean ±2 standard deviations).